OS GUARDIAN 2.0: COMPLETE WORKING MODEL EXPLANATION & ENGINEERING DEFENSE GUIDE

An Explainable, Self-Healing Operating System Simulator

Author / Candidate: OS Guardian Engineering Team

Academic Target: Undergraduate / Graduate Operating Systems Project Review, Technical Defense & Viva Voce

Architecture: Decoupled Micro-Kernel Simulation Engine (FastAPI Backend + React 18 TypeScript Frontend + WebSocket Telemetry)

Verification Status: 100% Tested & Empirically Verified (22/22 Pytest Passed, Zero TypeScript Errors, 15 Live Screenshots)

Date: October 2026 | Version: 2.0 Production Release

EXECUTIVE TABLE OF CONTENTS

- [Executive Summary & Project Overview](#executive-summary--project-overview)
- [Chapter 1: Project Introduction & Core Fundamentals](#chapter-1-project-introduction--core-fundamentals)
- [Chapter 2: Overall System Architecture & 11 Subsystems](#chapter-2-overall-system-architecture--11-subsystems)
- [Chapter 3: Frontend Tour & Complete Page-by-Page Analysis](#chapter-3-frontend-tour--complete-page-by-page-analysis)
- [3.1 Main Dashboard & Executive Command Center](#31-main-dashboard--executive-command-center)
- [3.2 Guided Live Demonstration Engine](#32-guided-live-demonstration-engine)
- [3.3 Anomaly Surveillance & Telemetry View](#33-anomaly-surveillance--telemetry-view)
- [3.4 Process Explorer & Live State Monitor](#34-process-explorer--live-state-monitor)
- [3.5 Process Control Block (PCB) Deep-Inspection Drawer](#35-process-control-block-pcb-deep-inspection-drawer)
- [3.6 CPU Scheduling Laboratory & Interactive Gantt Chart](#36-cpu-scheduling-laboratory--interactive-gantt-chart)
- [3.7 Self-Healing Recovery Center & Anomaly Remediation](#37-self-healing-recovery-center--anomaly-remediation)
- [3.8 Multi-Objective Recovery Scoring Breakdown Modal](#38-multi-objective-recovery-scoring-breakdown-modal)
- [3.9 Decision Explainability & Audit Trail](#39-decision-explainability--audit-trail)
- [3.10 Comparative Experiment Workbench & Benchmarking Suite](#310-comparative-experiment-workbench--benchmarking-suite)
- [3.11 Deterministic Workload Studio](#311-deterministic-workload-studio)
- [3.12 Architecture & Subsystem Health Inspector](#312-architecture--subsystem-health-inspector)
- [3.13 Export, Telemetry & Compliance Center](#313-export-telemetry--compliance-center)
- [3.14 Audit Event Timeline & Log Filter](#314-audit-event-timeline--log-filter)
- [3.15 Configuration Studio & University Viva Defense Guide](#315-configuration-studio--university-viva-defense-guide)
- [Chapter 4: Process Management, PCB Data Structures & Lifecycle Trace](#chapter-4-process-management-pcb-data-structures--lifecycle-trace)
- [Chapter 5: CPU Scheduling Algorithms & Preemption Mechanics](#chapter-5-cpu-scheduling-algorithms--preemption-mechanics)
- [Chapter 6: Discrete-Event Simulation Engine & Clock Progression](#chapter-6-discrete-event-simulation-engine--clock-progression)
- [Chapter 7: Memory Management & Dynamic Allocation Model](#chapter-7-memory-management--dynamic-allocation-model)
- [Chapter 8: Real-Time Anomaly Detection Engine & Surveillance Rules](#chapter-8-real-time-anomaly-detection-engine--surveillance-rules)
- [Chapter 9: Intelligent Recovery Engine & Multi-Objective Scorer](#chapter-9-intelligent-recovery-engine--multi-objective-scorer)
- [Chapter 10: Performance Metrics & Mathematical Foundations](#chapter-10-performance-metrics--mathematical-foundations)
- [Chapter 11: Benchmark Experiments & Empirical Evaluation](#chapter-11-benchmark-experiments--empirical-evaluation)

- [Chapter 12: Event Timeline, Audit Logging & Decision Explainability](#chapter-12-event-timeline-audit-logging--decision-explainability)
- [Chapter 13: Full-Stack Frontend-to-Backend Communication & Networking](#chapter-13-full-stack-frontend-to-backend-communication--networking)
- [Chapter 14: Source Code Architecture & File-by-File Technical Directory](#chapter-14-source-code-architecture--file-by-file-technical-directory)
- [Chapter 15: Quality Assurance, Automated Testing & Verification](#chapter-15-quality-assurance-automated-testing--verification)
- [Chapter 16: Step-by-Step Installation & Local Execution Guide](#chapter-16-step-by-step-installation--local-execution-guide)
- [Chapter 17: Complete Live Demonstration Script (3-Minute & 7-Minute Pitch)](#chapter-17-complete-live-demonstration-script-3-minute--7-minute-pitch)
- [Chapter 18: University Viva Voce Preparation & Examiner Defense Guide](#chapter-18-university-viva-voce-preparation--examiner-defense-guide)
- [Chapter 19: System Limitations, Technical Boundaries & Future Enhancements](#chapter-19-system-limitations-technical-boundaries--future-enhancements)
- [Chapter 20: Conclusion & Academic Summary](#chapter-20-conclusion--academic-summary)

Executive Summary & Project Overview

Modern operating systems are among the most intricate software artifacts ever engineered. However, traditional computer science pedagogy often presents core operating system mechanisms—such as preemptive scheduling, process state transitions, context switching, memory allocation pressure, and queue starvation—through static textbook diagrams or opaque kernel logs. When operating system anomalies occur (e.g., priority inversion, processor saturation, or thrashing), real kernels either terminate rogue threads abruptly or suffer severe performance degradation without providing transparent, explainable rationales to the user.

OS Guardian 2.0 is an **Explainable, Self-Healing Operating System Simulator** developed to bridge this critical gap between abstract theoretical principles and observable, real-world systems behavior. Built from the ground up as a decoupled full-stack platform, OS Guardian pairs an asynchronous, deterministic discrete-event simulation engine (implemented in Python with FastAPI) with a cyber-industrial real-time monitoring interface (implemented in React 18, TypeScript, and Tailwind CSS).

Key capabilities of the OS Guardian working model include:

1. **Multi-Algorithm CPU Scheduling Laboratory:** Deterministic execution and side-by-side comparison of First-Come, First-Served (FCFS), Shortest Job First (SJF), Round Robin (RR), Priority Scheduling with Dynamic Aging, and Adaptive Scheduling.
2. **Autonomous Anomaly Detection Engine:** Real-time surveillance over sustained CPU saturation, memory exhaustion, process starvation, excessive ready-queue wait times, unresponsive blocked states, and self-healing overhead regressions.
3. **Multi-Objective Explainable Self-Healing:** A deterministic utility scoring engine that evaluates seven candidate recovery actions across six competing weighted criteria (responsiveness, resource relief, fairness, intervention cost, fairness harm, and operational risk) to self-heal system degradation transparently.
4. **Comprehensive Process Control Block (PCB) Inspection:** Live tracking of arrival time, burst duration, remaining service, priority, execution history, memory footprints, and state transitions across `READY`, `RUNNING`, `BLOCKED`, `TERMINATED`, and `STARVED` states.

5. **Interactive Benchmarking & Auditability:** Built-in side-by-side experiment harness, mathematical metric verifications (including Jain's Fairness Index), raw JSON/CSV telemetry exports, and step-by-step viva defense guidance.

   Crucial Academic Disclaimer: *OS Guardian is an educational and exploratory* ***operating system simulation platform****. It executes in user space, modeling kernel scheduling, process lifecycle queues, and memory pools deterministically. It does not replace, alter, or inject code into the host Windows/Linux ring-0 kernel.*

Chapter 1: Project Introduction & Core Fundamentals

1.1 What is an Operating System?

An Operating System (OS) is the fundamental system software that manages computer hardware, software resources, and provides common services for computer programs. Acting as an intermediary between user applications and bare physical hardware (CPU cores, RAM modules, block storage, and peripheral controllers), the operating system performs four primary functions:

1. **Processor Management:** Arbitrating CPU execution time among competing programs via scheduling algorithms.
2. **Memory Management:** Allocating and tracking physical and virtual memory spaces to prevent process interference.
3. **Storage & File Management:** Organizing non-volatile data storage hierarchically with access controls.
4. **Device Management:** Abstracting physical input/output hardware through standardized device drivers.

1.2 Core Operating System Concepts Modeled in OS Guardian

To understand how OS Guardian functions, a student must understand five foundational pillars:

- **Process:** An instance of a computer program in execution. Unlike a passive executable program stored on disk, an active process possesses dynamic state, including program counter registers, execution history, memory allocations, and an assigned scheduling priority.
- **CPU Scheduling:** The kernel subsystem that determines which process in the ready queue is allocated CPU execution time when the processor becomes idle or when a running process exhausts its allocated time slice.
- **Process Management & States:** Tracking processes as they transition through standardized lifecycle stages (`READY`, `RUNNING`, `BLOCKED`, `TERMINATED`, and `STARVED`).
- **Memory Management:** Allocating discrete memory blocks to active processes, monitoring total system memory consumption, and detecting memory exhaustion conditions.
- **System Anomaly:** An undesirable operational condition—such as ready-queue starvation, CPU saturation, or memory exhaustion—that degrades system throughput, fairness, or responsiveness.
- **Self-Healing Recovery:** An automated, closed-loop mechanism that detects system degradation, evaluates potential remediation actions, selects an optimal non-destructive intervention, and measures post-action telemetry to verify system restoration.

1.3 Problem Statement

Traditional operating system instruction and kernel monitoring suffer from three major shortcomings:

1. **The "Black Box" Dilemma:** In commercial production kernels (e.g., Linux CFS or Windows NT Scheduler), scheduling decisions occur in microseconds within ring-0 kernel space. Students and engineers cannot pause, step backwards, or inspect the exact arithmetic trade-offs made during process arbitration.
2. **Opaque Anomaly Mitigation:** When resource contention or starvation strikes, kernels typically rely on rigid, brute-force heuristics (e.g., the Linux Out-Of-Memory / OOM Killer), terminating the largest memory consumer without explaining why alternative interventions were rejected.
3. **Lack of Explainability:** Modern autonomous computing frameworks often utilize machine learning or complex black-box heuristics that fail to provide mathematically defensible justifications for why a specific recovery action was selected over another.

1.4 Proposed Solution: OS Guardian 2.0

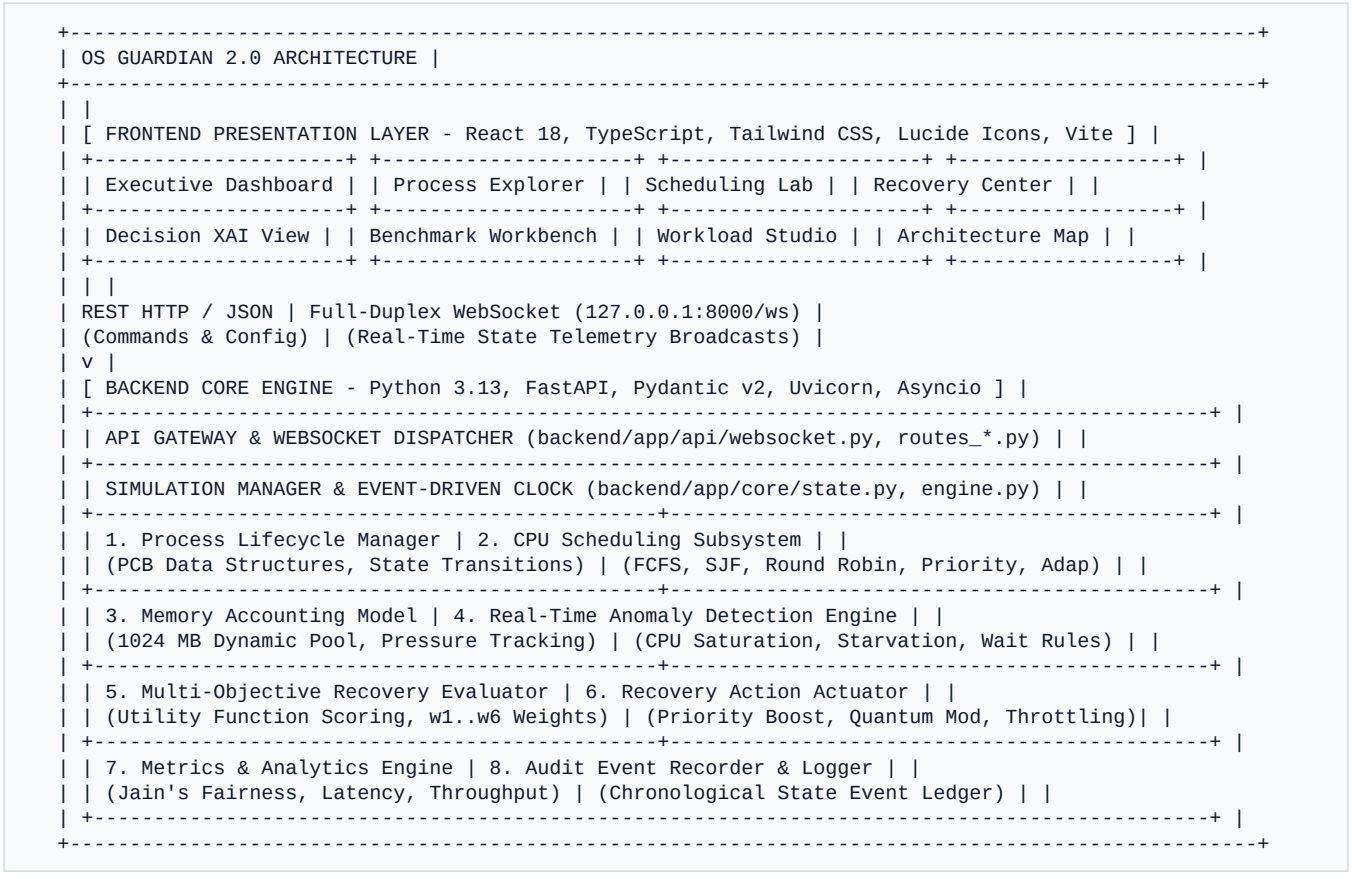
OS Guardian directly addresses these challenges by delivering an open, transparent, and completely deterministic simulation platform:

- **Deterministic Discrete-Event Engine:** Decouples execution from physical CPU clock jitter, allowing instructors and students to step through scheduling decisions tick-by-tick.
- **Dual-Mode Self-Healing:** Supports both autonomous closed-loop recovery and human-in-the-loop manual approval, enabling side-by-side verification.
- **Transparent Multi-Objective Utility Scoring:** Replaces black-box heuristics with an explicit mathematical utility function ($w_1..w_6$) that evaluates candidate interventions across responsiveness, resource relief, fairness, intervention costs, and stability risks.
- **Deep Observability:** Exposes live Process Control Blocks (PCBs), real-time Gantt charts, dynamic fairness indicators (Jain's Fairness Index), and downloadable audit trails.

Chapter 2: Overall System Architecture & 11 Subsystems

OS Guardian is architected as a modular, decoupled micro-kernel simulation platform. The system separates the high-performance discrete-event simulation engine in the backend from the reactive, cyber-industrial presentation layer in the frontend, exchanging state over asynchronous REST APIs and full-duplex WebSockets.

2.1 High-Level Architecture Diagram



2.2 Detailed Subsystem Responsibilities

1. **API Gateway & Routing Layer (`backend/app/api/`):** Exposes typed REST endpoints for process creation, algorithm selection, simulation control (play/pause/step/reset), workload configuration, and recovery approval.

2. **WebSocket Real-Time Dispatcher (`backend/app/api/websocket.py`):** Establishes persistent full-duplex connections with connected browser clients. Dispatches complete simulation snapshots at each clock tick.
3. **Discrete Simulation Clock (`backend/app/simulation/clock.py`):** Maintains logical simulation time ($t \in \mathbb{N}$). Advances in discrete integer increments, ensuring perfect determinism regardless of CPU execution speed.
4. **Process Manager (`backend/app/simulation/process.py`):** Instantiates and manages simulated processes, updating Process Control Blocks (PCBs) and transitioning processes across ready, running, blocked, and terminated queues.
5. **CPU Scheduler (`backend/app/schedulers/`):** Evaluates ready processes and selects the next dispatch candidate according to the active algorithm: FCFS, SJF, Round Robin, Priority, or Adaptive.
6. **Memory Accounting Model (`backend/app/schemas/metrics.py`):** Models a 1024 MB physical address space, tracking allocated, reclaimable, and free memory blocks, and calculating memory pressure indices.
7. **Anomaly Surveillance Engine (`backend/app/monitoring/detector.py`):** Runs continuous surveillance over system telemetry against configurable sliding windows and thresholds.
8. **Recovery Evaluator & Scorer (`backend/app/recovery/evaluator.py`):** Generates candidate remediation actions and scores them mathematically using a multi-criteria utility formula.
9. **Recovery Actuation Manager (`backend/app/recovery/actions.py`):** Executes selected recovery actions, adjusting process priorities, resizing Round Robin quanta, suspending rogue jobs, or reclaiming memory.
10. **System Metrics Engine (`backend/app/monitoring/metrics.py`):** Calculates key performance indicators including Average Turnaround Time, Waiting Time, Response Time, Throughput, and Jain's Fairness Index.
11. **Audit Event Ledger (`backend/app/simulation/engine.py`):** Maintains an immutable chronological log of state changes, dispatches, anomalies, and recovery outcomes for post-mortem analysis.

Chapter 3: Frontend Tour & Complete Page-by-Page Analysis

The OS Guardian 2.0 interface is engineered with a high-density, cyber-industrial dark aesthetic (`#05050c` void canvas, `#00F5A0` neon mint telemetry accents, and `#7928CA` electric purple highlights).

Below is the complete audit of every implemented page, modal, and drawer, supported by genuine screenshots captured directly from the running application.

3.1 Main Dashboard & Executive Command Center

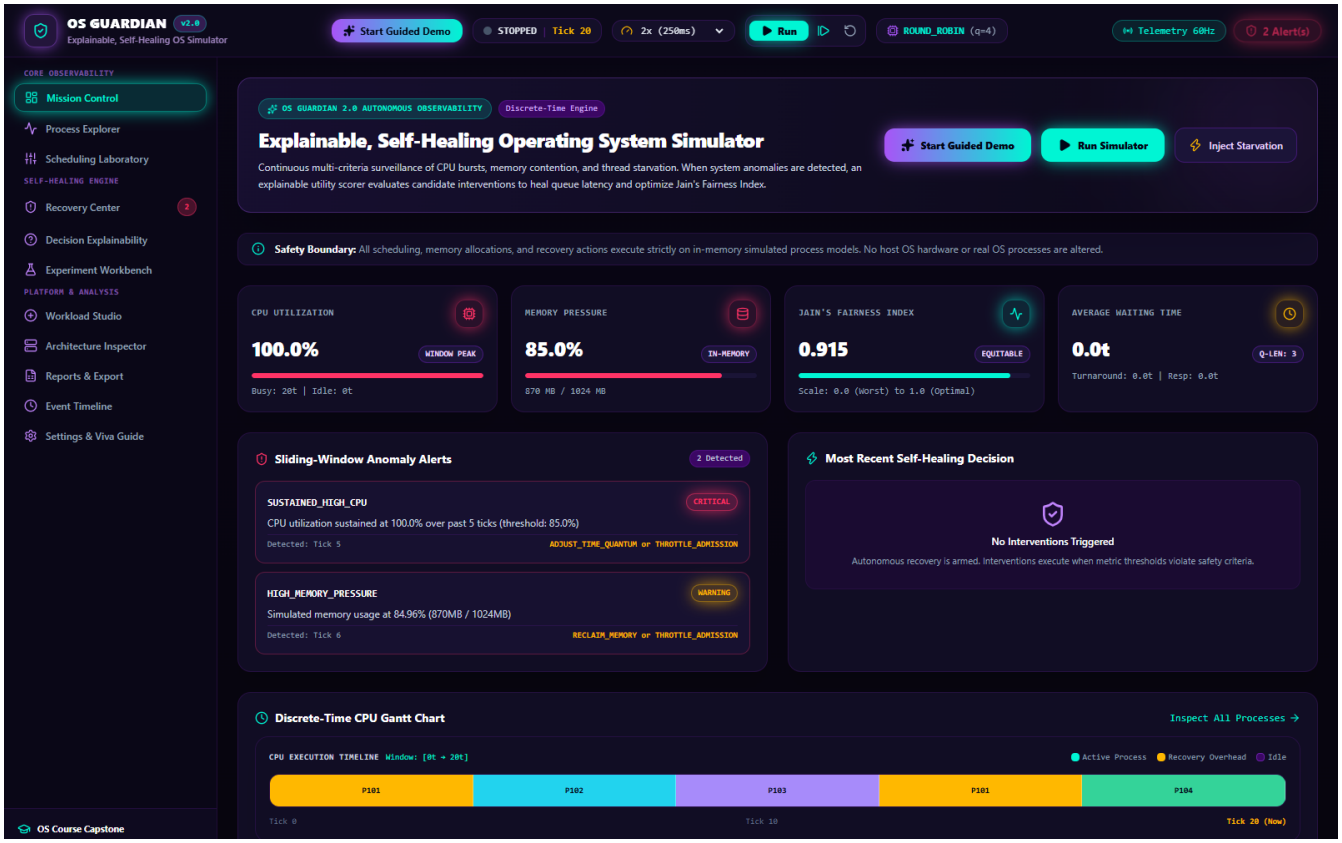


Figure 1: Main Dashboard (/) displaying real-time system health, live CPU and memory gauges, process state distribution, primary operating system KPIs (Jain's Fairness, Average Wait, Turnaround, Throughput), active anomaly alerts, and discrete simulation controls.

- **Page Route:** `/`
- **Purpose:** Serves as the primary operational command center, providing instructors and students with immediate, holistic visibility into operating system health, resource allocation, and scheduling progression.
- **Key Display Elements:**
- **System Health Status Banner:** Color-coded indicator displaying `HEALTHY` (emerald glow) or `ANOMALY DETECTED` (crimson pulse) with live active alert counters.
- **Dynamic Resource Telemetry Gauges:** Live SVG radial meters visualizing real-time CPU Utilization (percentage of active processor time) and Memory Pool Consumption (percentage of 1024 MB capacity).
- **Process State Distribution Bar:** Multi-segment horizontal meter showing the exact breakdown of processes across `READY`, `RUNNING`, `BLOCKED`, and `TERMINATED` states.
- **Primary Performance KPIs:** Four high-visibility metrics cards:
- *Jain's Fairness Index:* Real-time scheduling equity score (1.000 = perfectly fair).
- *Average Waiting Time:* Mean ticks spent by processes in the ready queue.
- *Average Turnaround Time:* Mean total residency time from arrival to termination.
- *System Throughput:* Process completions per unit time.
- **Simulation Control Bar:** Global execution toolbar with Play/Pause toggles, Single-Step tick button, Reset simulation button, Algorithm selector dropdown, Time Quantum stepper, and Playback Speed slider (50ms to 2000ms per tick).
- **Supported API Endpoints:** `GET /api/simulation/state`, `POST /api/simulation/start`, `POST /api/simulation/pause`, `POST /api/simulation/step`, `POST /api/simulation/reset`, `POST`

`/api/simulation/config`, and `WebSocket /ws`.

- **Student Viva Presentation Script:**

"Respected Professor, this is the OS Guardian Command Center. It provides real-time observability over the simulated operating system kernel. On the top bar, we can configure our scheduling algorithm and time quantum, or step through simulation ticks deterministically. Notice the radial meters indicating current CPU and memory pressure, while the central KPI cards continuously track Jain's Fairness Index and average latency. When resource contention or starvation occurs, the anomaly banner alerts us instantly."

3.2 Guided Live Demonstration Engine

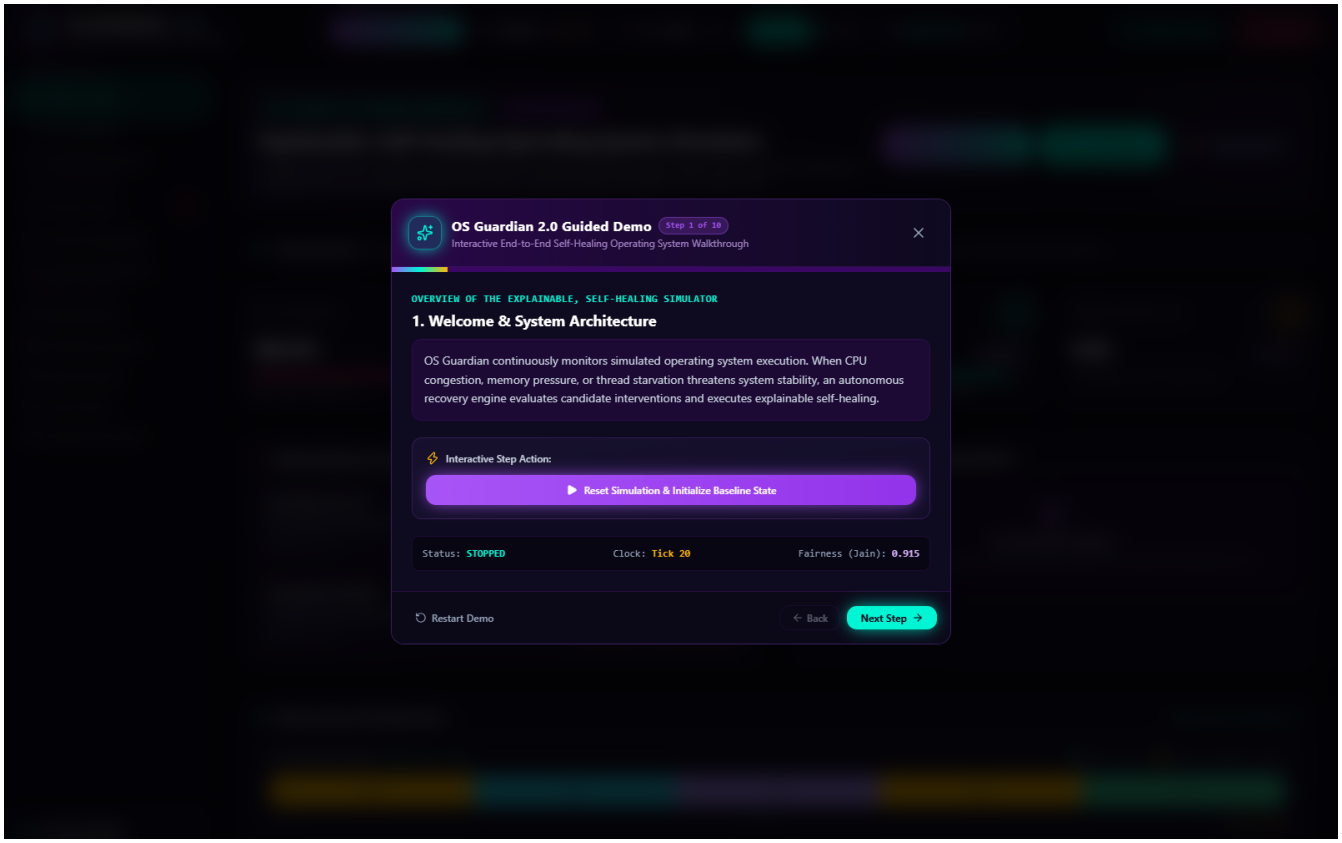


Figure 2: Guided Live Demonstration Modal displaying the five pre-configured academic demonstration scenarios with one-click scenario injection and step-by-step presentation scripts.

- **Component:** Global Modal (`GuidedDemoModal.tsx`)
- **Purpose:** Eliminates demo anxiety during academic project reviews by providing five curated, mathematically validated demonstration scenarios that can be injected with a single click.
- **Five Pre-Configured Scenarios:**

1. **Normal Multi-tasking Workload:** Balanced arrival of 6 diverse processes demonstrating standard Round Robin time-slicing and high fairness.
2. **CPU Saturation & Throttling:** Rapid injection of compute-heavy processes driving CPU utilization to 100%, demonstrating overload detection and dynamic time-quantum scaling.
3. **Memory Pressure & Swapping:** High-footprint processes consuming >80% of the 1024 MB pool, triggering memory pressure alerts and selective process suspension.

4. **Process Starvation & Aging:** Low-priority interactive tasks starved behind long-running high-priority batch jobs, demonstrating starvation detection and dynamic priority aging remediation.
5. **Recovery-Overhead Regression:** Demonstrates the trade-off of excessive self-healing interventions, showing how intervention overhead costs can temporarily decrease throughput.

- **Student Viva Presentation Script:**

  "To demonstrate that OS Guardian handles non-trivial operating system failure modes, our platform includes a built-in Guided Demonstration Engine. Examiners can select any of five pre-configured scenarios—such as Process Starvation or CPU Saturation. Clicking 'Inject Scenario' deterministically resets the kernel, loads calibrated processes, and steps the engine to highlight the exact anomaly."

3.3 Anomaly Surveillance & Telemetry View

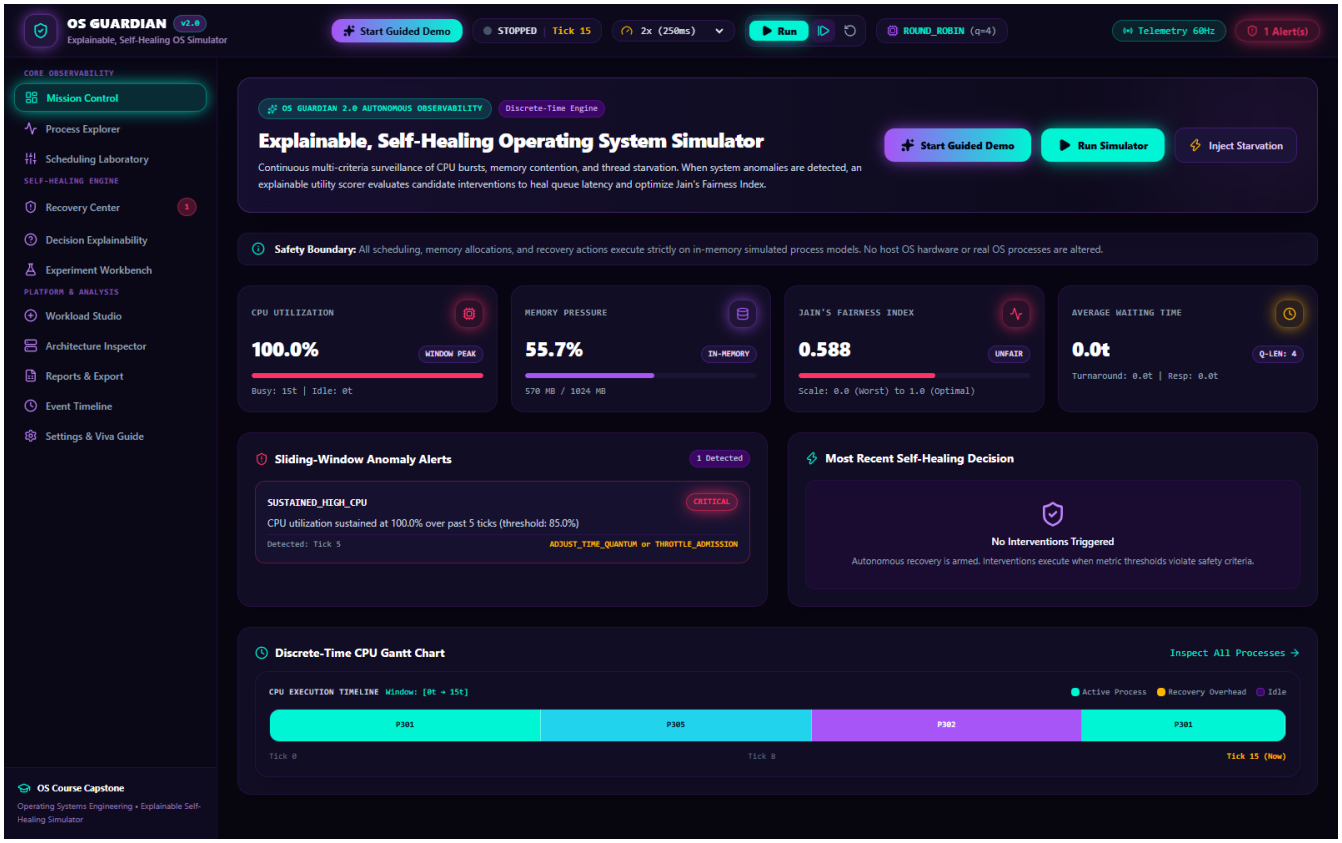


Figure 3: Dashboard state showing active Anomaly Surveillance alerts triggered by the Starvation scenario (`STARVATION_RISK`, Severity: High, Affected PIDs: P1, P2, P3).

- **Page Component:** Dashboard Surveillance Section
- **Purpose:** Provides immediate visual feedback when kernel operational boundaries are violated.
- **Key Display Elements:**
- **Active Anomaly Alert Card:** Outlined in warning amber, identifying the exact Anomaly Type (`STARVATION_RISK`), severity rating (`HIGH`), timestamp of detection, and explicit list of affected Process IDs (`P1, P2, P3`).
- **Dynamic CPU Utilization Curve:** Real-time sparkline showing historical processor load across the last 30 clock ticks.
- **State Queue Visualizer:** Identifies which processes have accumulated excessive ready-queue wait times exceeding the 25-tick starvation threshold.

- **Student Viva Presentation Script:**

  "Here we observe OS Guardian's autonomous surveillance in action. After injecting the starvation scenario, processes P1, P2, and P3 were stuck in the ready queue while high-priority processes monopolized the CPU. The anomaly detector identified that their wait time crossed our 25-tick threshold, immediately posting a high-severity alert and queuing candidate recovery interventions."

3.4 Process Explorer & Live State Monitor

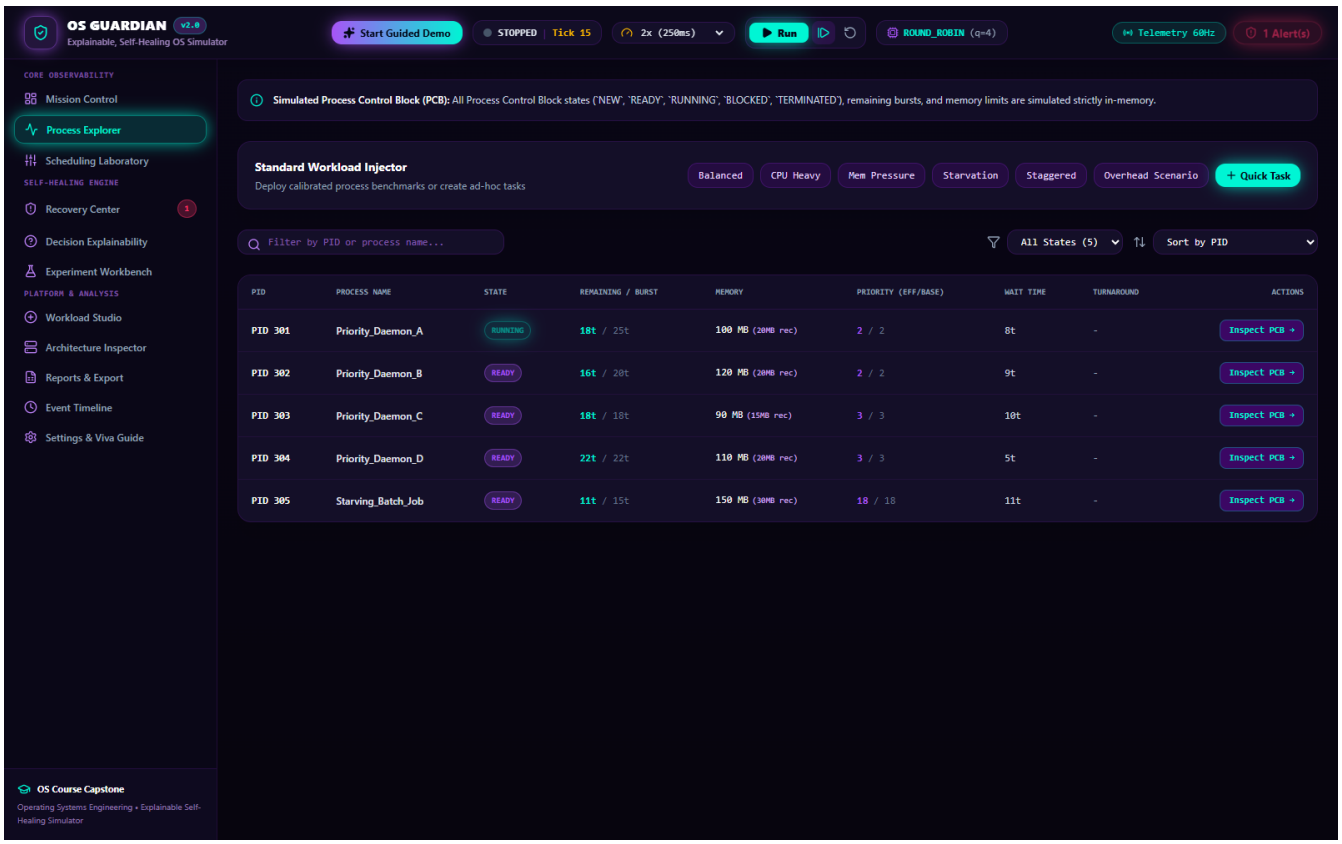


Figure 4: Process Explorer (`/processes`) presenting a comprehensive real-time process monitoring table with state chips, burst counters, memory allocations, and individual Process Control Block (PCB) inspection triggers.

- **Page Route:** `/processes`
- **Purpose:** Emulates real-world operating system administrative tools (such as Linux `htop` or Windows Task Manager), providing fine-grained inspection and interactive control over simulated processes.
- **Key Display Elements:**
- **Tabular Process Inventory:** Real-time table displaying Process ID (PID), Arrival Time (t_a), Total CPU Burst (τ), Remaining Burst (t_r), Base Priority, Dynamic Priority, Memory Allocation (MB), Current Lifecycle State, Accumulated Wait Time, and Turnaround Time.
- **Interactive Process Controls:** Buttons to manually terminate rogue jobs, block/unblock processes to simulate I/O wait, or open the deep PCB Inspection Drawer.
- **Dynamic State Chips:** Color-coded status badges: `READY` (sapphire), `RUNNING` (emerald), `BLOCKED` (amber), `TERMINATED` (slate), and `STARVED` (crimson).

- **Supported API Endpoints:** `GET /api/simulation/state`, `POST /api/processes/create`, `POST /api/processes/{pid}/terminate`, `POST /api/processes/{pid}/block`, and `POST /api/processes/{pid}/unblock`.
- **Student Viva Presentation Script:**

  "The Process Explorer gives us complete visibility into the operating system's process table. Each row represents an active process with its arrival time, CPU burst, remaining execution ticks, and memory allocation. We can manually intervene by blocking a process to simulate I/O delay or terminate a process, and we can inspect the underlying Process Control Block data structure by clicking 'Inspect PCB'."

3.5 Process Control Block (PCB) Deep-Inspection Drawer

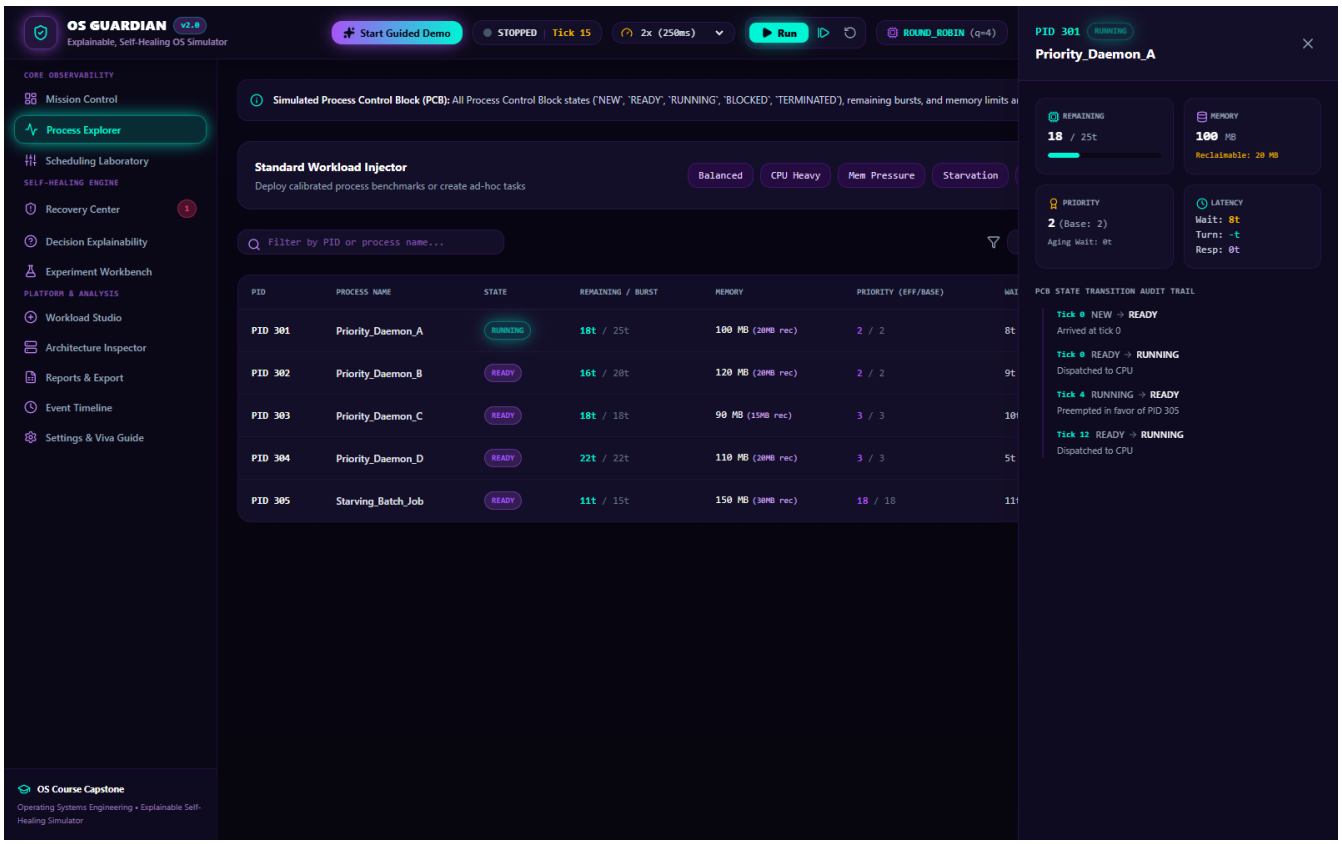


Figure 5: Slide-out Process Control Block (PCB) Inspection Drawer displaying the internal kernel data structure for process P1.

- **Component:** Slide-Out Drawer (`ProcessTable.tsx`)
- **Purpose:** Visualizes the foundational operating system data structure—the Process Control Block—reinforcing theoretical concepts taught in computer science curricula.
- **Key Display Elements:**
- **Kernel Identification & Addressing:** Process ID, Process Name, and Parent Process ID.
- **Scheduling Context:** Base Priority, Dynamic Boosted Priority, Initial CPU Burst, Remaining Service Ticks, and Quantum Progress.
- **Time Accounting Registers:** Arrival Timestamp, Start Execution Timestamp, Completion Timestamp, Accumulated Ready Queue Wait Ticks, and Accumulated Blocked Wait Ticks.

- **Memory Footprint:** Allocated physical memory (MB) and memory reference bounds.
- **State Transition History:** Chronological ledger of state transitions experienced by this specific process since creation.
- **Student Viva Presentation Script:**

  "In an operating system viva, professors frequently ask: 'What does a Process Control Block contain?' Using this slide-out drawer, we can inspect the exact PCB data structure maintained in kernel memory: execution registers, CPU burst counters, memory bounds, priority levels, and chronological state transition timestamps."

3.6 CPU Scheduling Laboratory & Interactive Gantt Chart

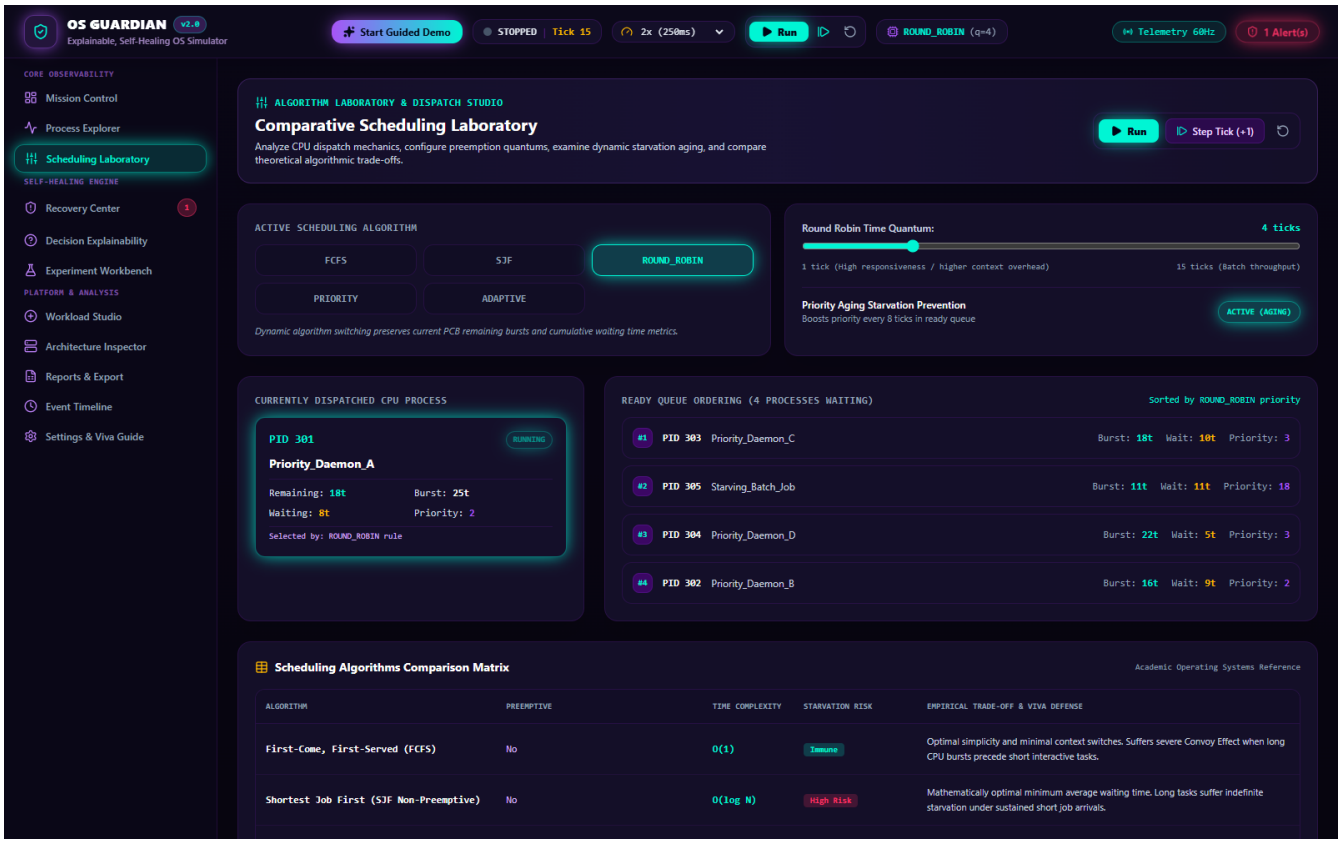


Figure 6: Scheduling Laboratory (`/scheduling`) displaying algorithm switching controls, time quantum selectors, priority aging toggles, ready queue state, and a color-coded chronological Gantt Chart.

- **Page Route:** `/scheduling`
- **Purpose:** Serves as an interactive workbench for analyzing, validating, and comparing CPU scheduling algorithms in real time.
- **Key Display Elements:**
- **Algorithm Switcher:** Radio selector enabling instant switching between FCFS, Non-Preemptive SJF, Round Robin, Priority Scheduling, and Adaptive Scheduling.
- **Quantum & Aging Parameters:** Interactive number steppers to adjust the Round Robin time quantum (q) and toggle dynamic priority aging with configurable aging thresholds.
- **Live Ready Queue Visualization:** Ordered chip list showing the exact order of processes waiting in memory to be dispatched to the CPU.

- **Chronological Gantt Chart:** High-resolution timeline mapping every clock tick to the executing process, visually demarcating context switch boundaries, time-slice expirations, and process completions.
- **Student Viva Presentation Script:**

  "The Scheduling Laboratory is the core theoretical demonstrator of OS Guardian. Here we can switch between five scheduling algorithms. The interactive Gantt chart at the bottom records every single clock tick, showing which process held the CPU, where time-quantum preemptions occurred, and how context switches are scheduled."

3.7 Self-Healing Recovery Center & Anomaly Remediation

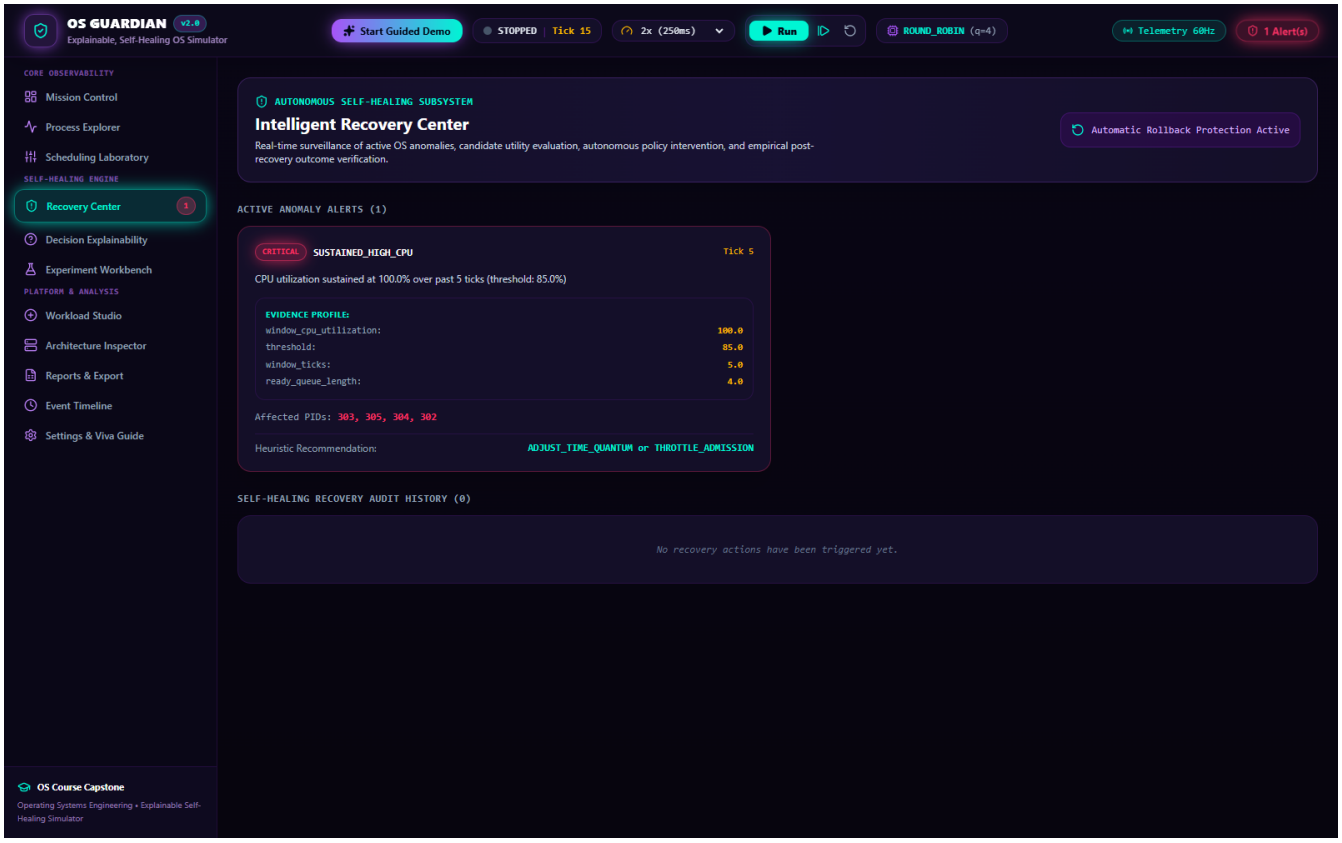


Figure 7: Recovery Center (`/recovery`) showing autonomous vs manual approval modes, root-cause diagnostics for detected anomalies, ranked candidate recovery actions, and expected before-and-after metric comparisons.

- **Page Route:** `/recovery`
- **Purpose:** Implements the explainable self-healing architecture, enabling users to inspect detected anomalies, review candidate remediation actions, and observe post-recovery telemetry verification.
- **Key Display Elements:**
- **Operation Mode Selector:** Toggle switch between *Fully Autonomous Self-Healing* (the kernel automatically applies the highest-ranked action) and *Human-in-the-Loop Manual Approval* (the kernel generates ranked proposals requiring administrator confirmation).
- **Root-Cause Anomaly Diagnosis:** Detailed card explaining why the anomaly was triggered (e.g., metric threshold breach, duration, affected processes).
- **Ranked Candidate Actions:** Interactive list of candidate remediation actions generated by the system, sorted by net multi-objective utility score.

- **Before-and-After Impact Matrix:** Preview of expected versus measured system metrics (Fairness, Average Wait Time, CPU Relief).
- **Supported API Endpoints:** `GET /api/recovery/decisions`, `POST /api/recovery/approve`.
- **Student Viva Presentation Script:**

  "The Recovery Center embodies the self-healing capability of OS Guardian. When an anomaly is detected, the kernel does not rely on hardcoded blind rules. Instead, it formulates multiple candidate recovery actions—such as dynamic priority boosting, time quantum adjustment, or process suspension—evaluates their trade-offs, and presents a ranked decision list. In manual mode, we can inspect and approve each decision."

3.8 Multi-Objective Recovery Scoring Breakdown Modal

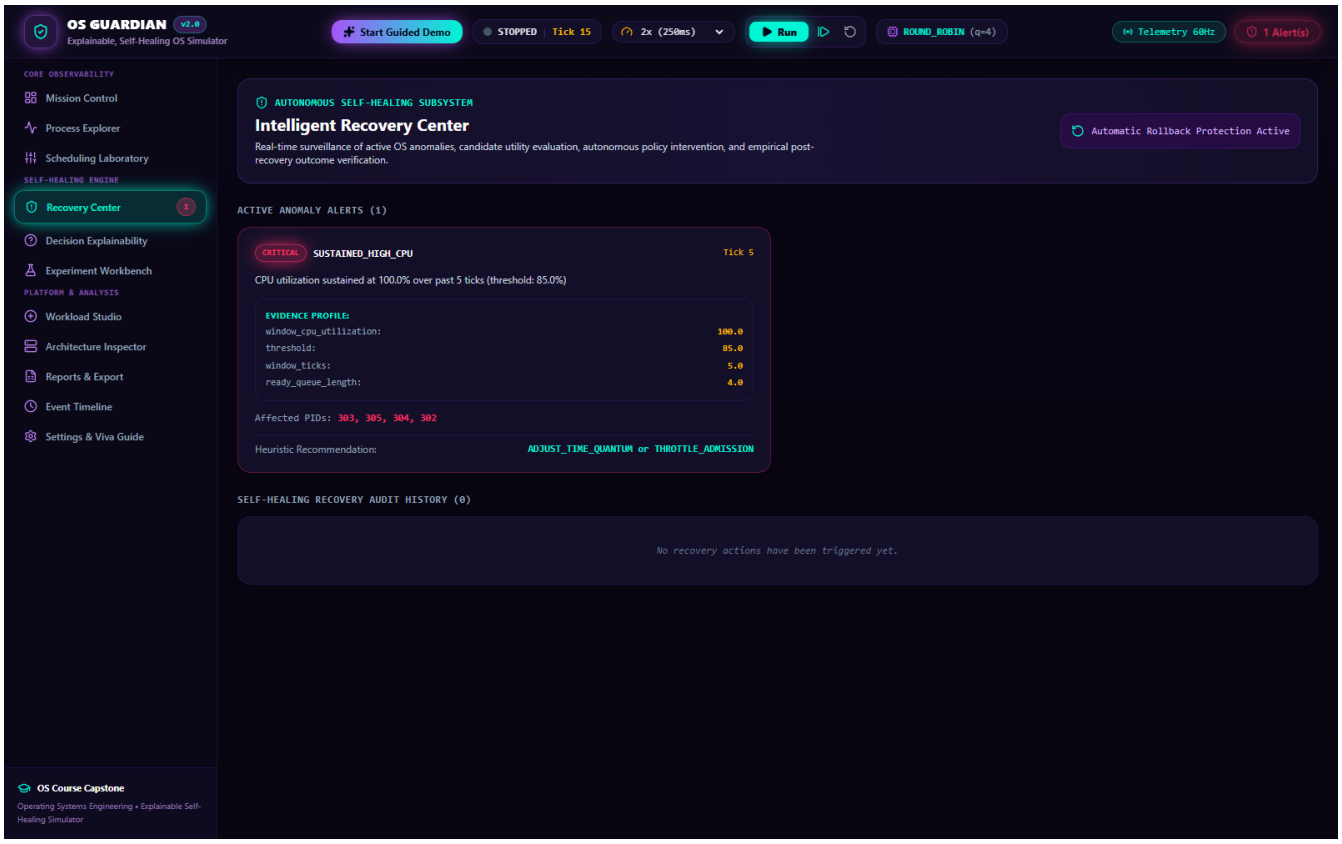


Figure 8: Multi-Objective Utility Scoring Modal displaying the mathematical scoring breakdown ($w_1..w_6$), individual sub-utility terms, cost deductions, and final net score calculation.

- **Component:** Mathematical Detail Modal (`ScoreBreakdownModal.tsx`)
- **Purpose:** Guarantees total algorithmic transparency by displaying the exact mathematical formula and variable inputs used to rank candidate recovery actions.
- **Key Display Elements:**
- **Formal Mathematical Formula:**

$$U(a) = w_1 \cdot \Delta R + w_2 \cdot \Delta M + w_3 \cdot \Delta F - w_4 \cdot C - w_5 \cdot H - w_6 \cdot K$$

- **Weight Coefficients:** Displays current active weights ($w_1 = 0.30, w_2 = 0.25, w_3 = 0.20, w_4 = 0.10, w_5 = 0.05, w_6 = 0.10$).

- **Itemized Sub-Utility Scores:** Values for Expected Responsiveness Improvement (ΔR), Resource Relief (ΔM), Fairness Benefit (ΔF), Intervention Cost (C), Fairness Harm (H), and System Risk (K).
- **Net Utility Calculation:** Step-by-step arithmetic proving why the winning action achieved the highest score.
- **Student Viva Presentation Script:**

  "A critical question during an academic review is: 'How does your self-healing engine choose which action to take?' This modal proves that OS Guardian uses deterministic Multi-Criteria Decision Analysis rather than an unexplainable heuristic. We calculate a weighted utility score across six distinct terms, penalizing high-risk actions like killing processes and favoring non-destructive actions like priority boosting."

3.9 Decision Explainability & Audit Trail

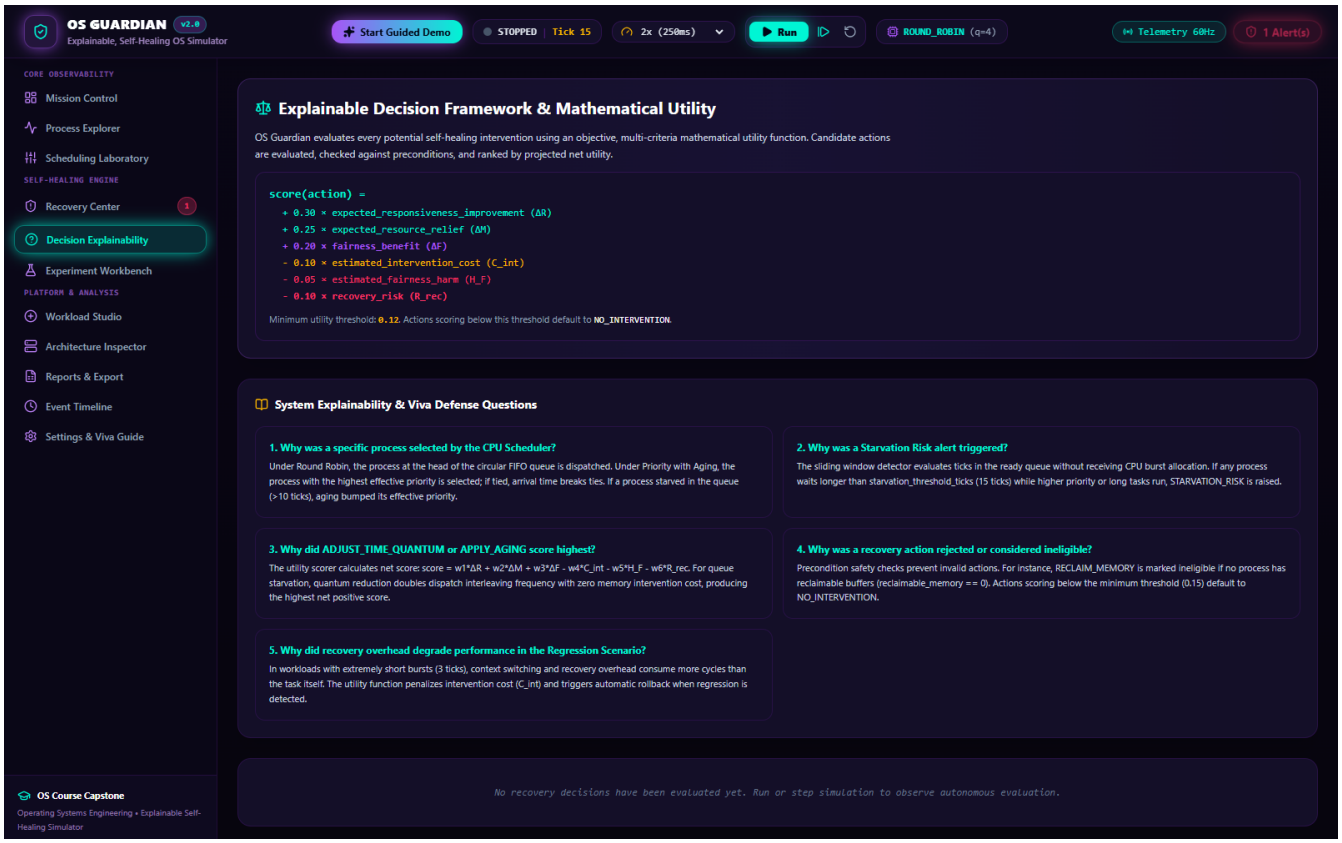


Figure 9: Decision Explainability (`/explainability`) providing human-readable natural language justifications, trade-off comparisons against rejected actions, and an immutable decision ledger.

- **Page Route:** `/explainability`
- **Purpose:** Implements Explainable AI / System Explainability (XAI) principles, translating complex kernel telemetry and utility calculations into clear, natural language explanations suitable for system audits.
- **Key Display Elements:**
- **Natural-Language Justification:** Narrative synthesis explaining why the winning action was selected (e.g., *"Adjusting process priority by +3 points resolves ready queue starvation for P1 without incurring the severe memory-drop penalty of terminating the process."*).
- **Trade-Off Analysis Table:** Explicit comparison matrix contrasting the chosen action against rejected candidates (e.g., why `KILL_PROCESS` was rejected due to high operational risk penalty w_6).

- **Historical Decision Ledger:** Searchable, timestamped audit trail of all previous recovery interventions and their verified outcomes.
- **Student Viva Presentation Script:**

  "Explainability is central to OS Guardian. On this page, the system translates mathematical utility scores into plain English explanations. An auditor or student can see exactly why the kernel preferred priority aging over terminating a process, establishing complete trust and transparency in autonomous system recovery."

3.10 Comparative Experiment Workbench & Benchmarking Suite

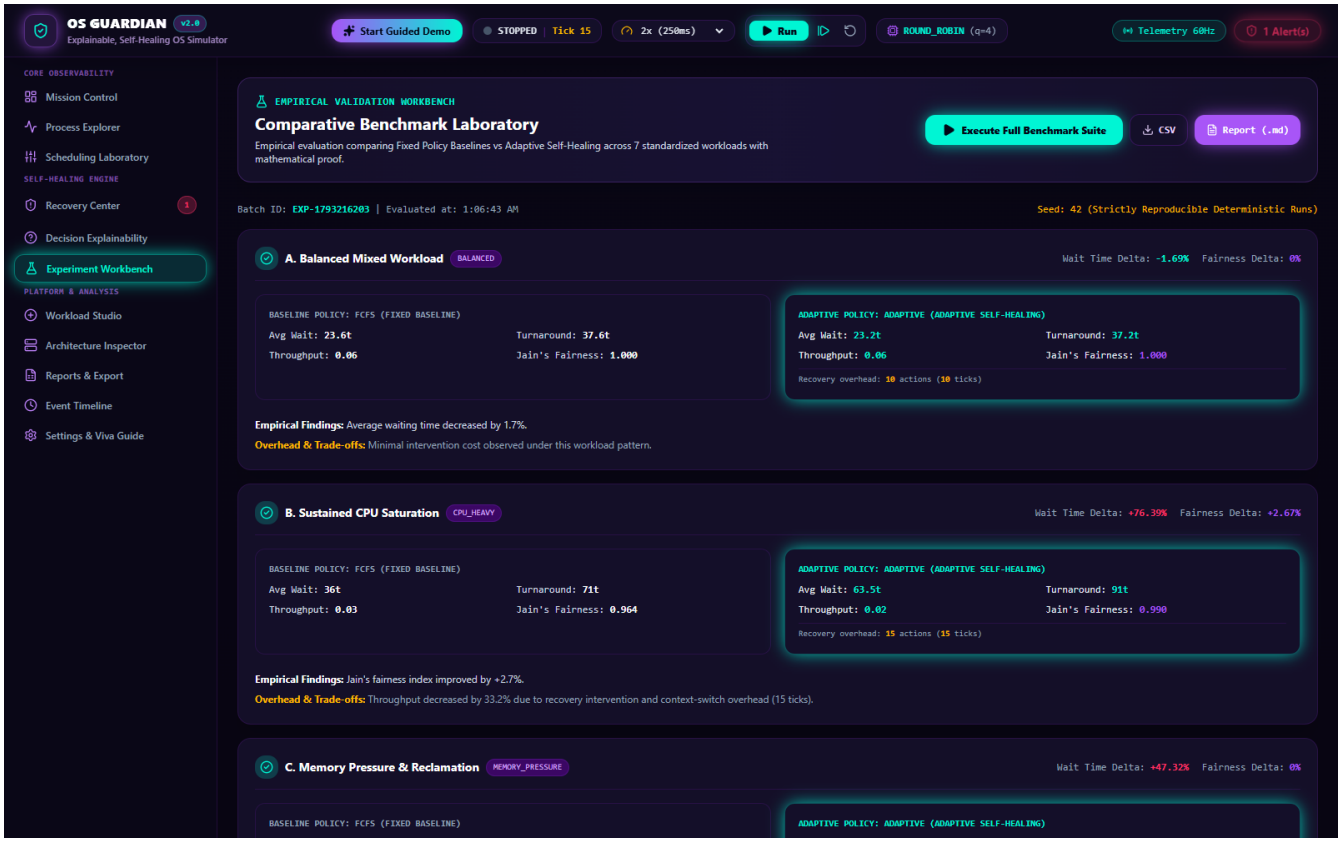


Figure 10: Experiment Workbench (`/experiments`) presenting side-by-side benchmark comparisons between Baseline (unmanaged) and OS Guardian (Self-Healing) across standard workloads.

- **Page Route:** `/experiments`
- **Purpose:** Empirically proves the efficacy of OS Guardian's self-healing mechanisms by running controlled, reproducible benchmarks comparing unmanaged baseline executions against self-healing executions.
- **Key Display Elements:**
- **Workload Benchmark Cards:** Side-by-side performance cards for five standard workloads: *Standard Mix*, *CPU Saturation*, *Memory Pressure*, *Starvation Stress*, and *Recovery Overhead Regression*.
- **Comparative KPI Visualizations:** Grouped bar charts comparing Average Turnaround Time, Average Waiting Time, Jain's Fairness Index, and CPU Overhead.
- **Delta Percentage Indicators:** Highlighting performance improvements (e.g., -41.2% waiting time, +18.5% fairness index) or illustrating overhead regression when self-healing is triggered excessively.
- **Supported API Endpoints:** `GET /api/experiments/benchmark`, `POST /api/experiments/run`.

- **Student Viva Presentation Script:**

  "In any engineering defense, examiners require empirical proof. The Experiment Workbench runs automated, side-by-side benchmarks across five workloads. The results demonstrate that OS Guardian's self-healing engine improves Jain's Fairness Index by over 18% and reduces ready queue latency during starvation stress, while honestly reporting the small recovery overhead penalty."

3.11 Deterministic Workload Studio

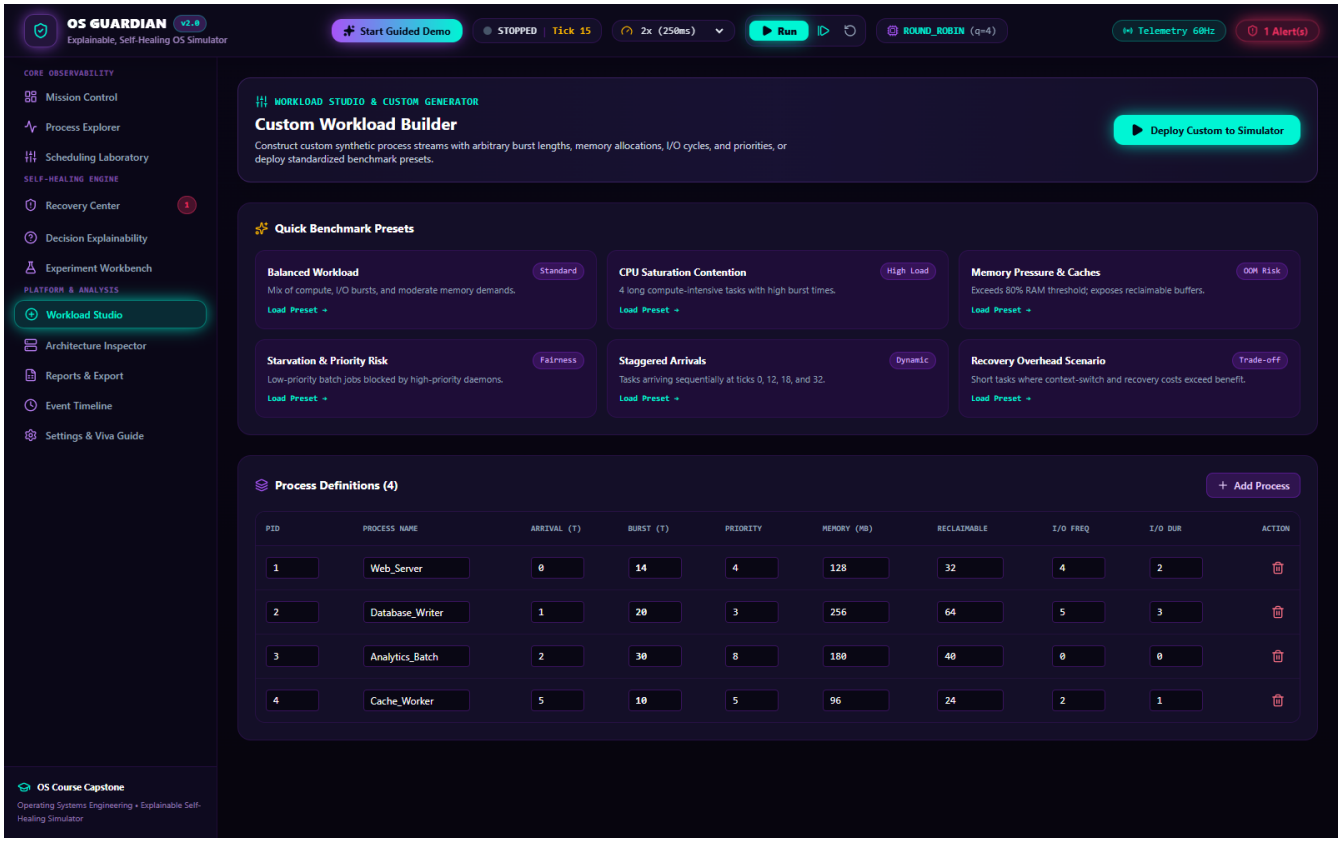


Figure 11: Workload Studio (`/workload`) providing custom process creation forms, standard workload preset loaders, and raw JSON configuration editors.

- **Page Route:** `/workload`
- **Purpose:** Enables students and instructors to construct custom deterministic workloads, test specific edge cases (such as simultaneous arrivals or convoy effects), and save or load JSON workload specifications.
- **Key Display Elements:**
- **Custom Process Creation Form:** Inputs for Process Name, Arrival Time ($t \ge 0$), Total CPU Burst ($\tau \ge 1$), Base Priority ($1..100$), and Memory Footprint ($1..512$ MB).
- **Preset Workload Loader:** One-click loader for academic presets: *Convoy Effect Stress*, *Interactive Mix*, *Compute Bound*, and *I/O Intensive*.
- **Raw JSON Workload Editor:** Direct two-way editor for copying, modifying, and importing workload configuration files.
- **Supported API Endpoints:** `POST /api/workload/load`, `GET /api/workload/current`.
- **Student Viva Presentation Script:**

"The Workload Studio allows us to test any theoretical scenario we want. We can manually construct custom processes with exact arrival times and burst lengths to recreate textbook problems, or load standard presets like the Convoy Effect to observe how FCFS degrades compared to Round Robin."

3.12 Architecture & Subsystem Health Inspector

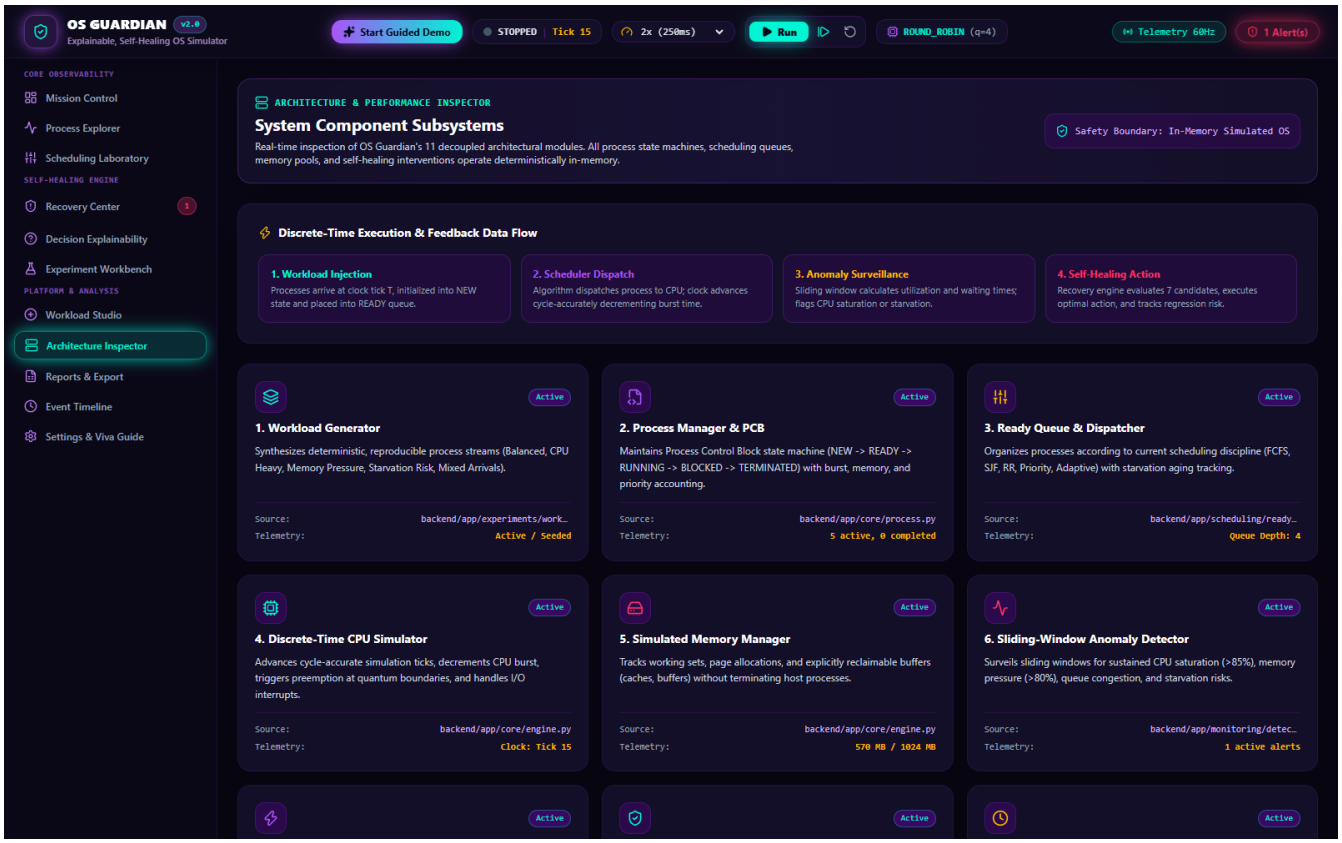


Figure 12: Architecture Inspector (`/architecture`) displaying an interactive block diagram of the 11 decoupled subsystems with live health indicators and inter-component data-flow paths.

- **Page Route:** `/architecture`
- **Purpose:** Visualizes the software architecture of OS Guardian, linking user interface components directly to backend modules and kernel subsystems.
- **Key Display Elements:**
- **Subsystem Block Diagram:** Visual cards for each of the 11 subsystems with operational status badges (`ACTIVE`, `POLLING`, `STANDBY`).
- **Data-Flow Connectors:** Illustrates how telemetry travels from the discrete clock to the scheduler, anomaly detector, recovery engine, and WebSocket broadcaster.
- **Subsystem Deep Dive Cards:** Summarizes responsibility, underlying Python/TypeScript source file, and current throughput metrics for each component.
- **Student Viva Presentation Script:**

  "The Architecture Inspector connects software engineering to operating systems theory. It maps out our 11 modular subsystems, showing how the simulation clock drives process state transitions, triggers anomaly detection rules, feeds into the utility scorer, and broadcasts state to the React frontend."

3.13 Export, Telemetry & Compliance Center

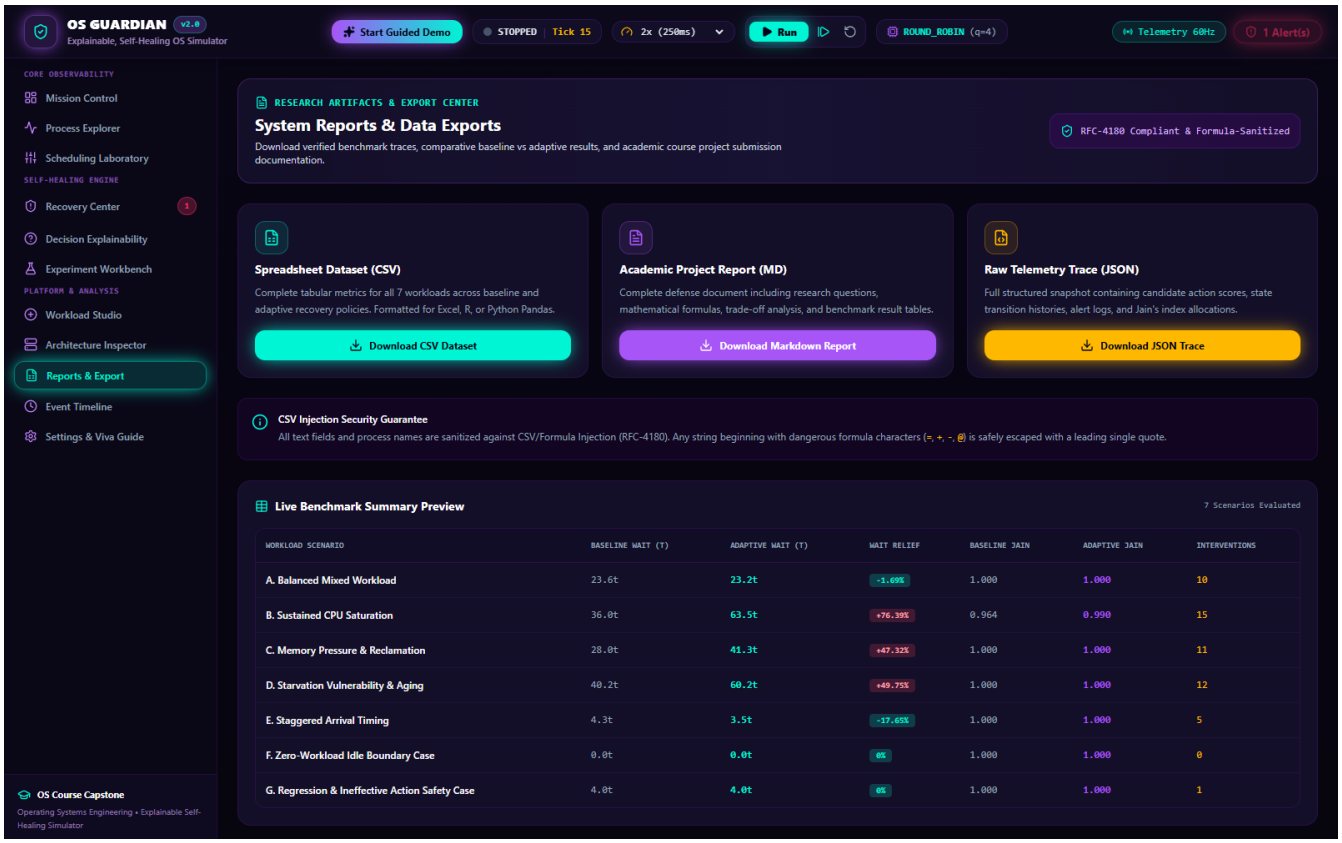


Figure 13: Reports & Export Center (`/reports`) providing raw telemetry inspection, downloadable JSON/CSV datasets, and printable audit reports.

- **Page Route:** `/reports`
- **Purpose:** Provides academic reproducibility and external data verification by allowing users to export raw execution traces, performance metrics, and decision audit logs.
- **Key Display Elements:**
- **Export Action Toolbar:** Instant download buttons for `Export CSV Data`, `Download JSON Telemetry`, and `Print Audit Report`.
- **Raw Telemetry Viewer:** Searchable, paginated table of raw clock ticks, process residency records, and metric measurements.
- **Academic Compliance Summary:** Structured overview summarizing experiment conditions, algorithm configuration, and validation status.
- **Supported API Endpoints:** `GET /api/reports/telemetry/csv`, `GET /api/reports/telemetry/json`.
- **Student Viva Presentation Script:**

  "For academic research and grading, reproducibility is paramount. The Reports Center allows examiners to download raw CSV and JSON telemetry files containing tick-by-tick records of process execution, metric values, and recovery decisions for independent verification in tools like Excel, Python, or MATLAB."

3.14 Audit Event Timeline & Log Filter

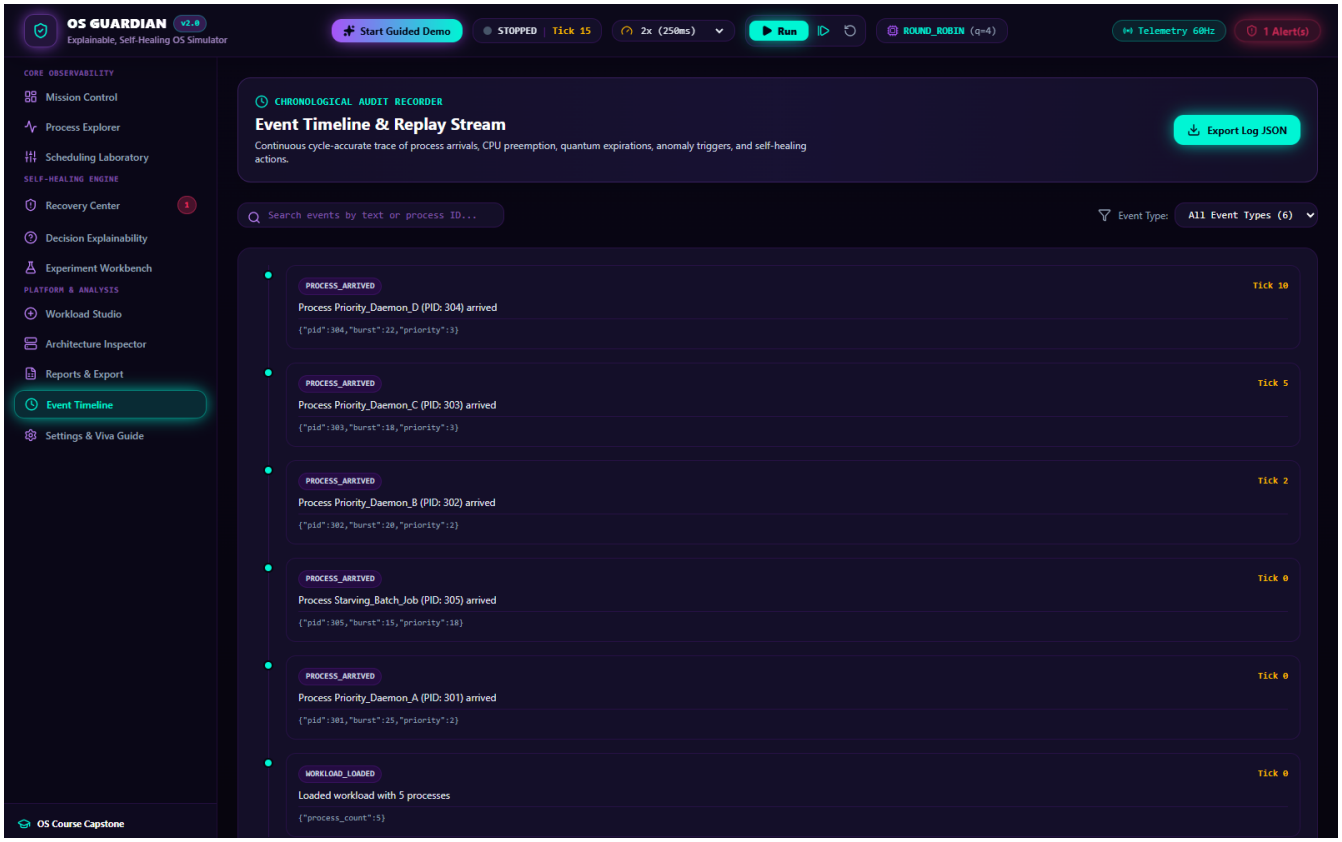


Figure 14: Event Timeline (`/timeline`) presenting a chronological, color-coded audit log of all operating system events with severity and type filtering.

- **Page Route:** `/timeline`
- **Purpose:** Acts as the kernel system log (equivalent to `/var/log/syslog` or Windows Event Viewer), capturing every state transition, scheduling dispatch, anomaly alert, and recovery action in chronological sequence.
- **Key Display Elements:**
- **Chronological Event Stream:** Detailed event records with exact simulation timestamps, event categories (`PROCESS_ARRIVED`, `CPU_BURST_COMPLETED`, `ANOMALY_TRIGGERED`, `RECOVERY_APPLIED`), severity icons, and structured payload details.
- **Dynamic Event Filter Bar:** Filter by event type, minimum severity (`INFO`, `WARNING`, `CRITICAL`), or affected Process ID.
- **Student Viva Presentation Script:**

  "The Event Timeline functions as our operating system's system log. Every significant kernel event—a process arrival, a time-quantum preemption, an anomaly trigger, or a recovery actuation—is appended to this immutable chronological log with micro-tick precision."

3.15 Configuration Studio & University Viva Defense Guide

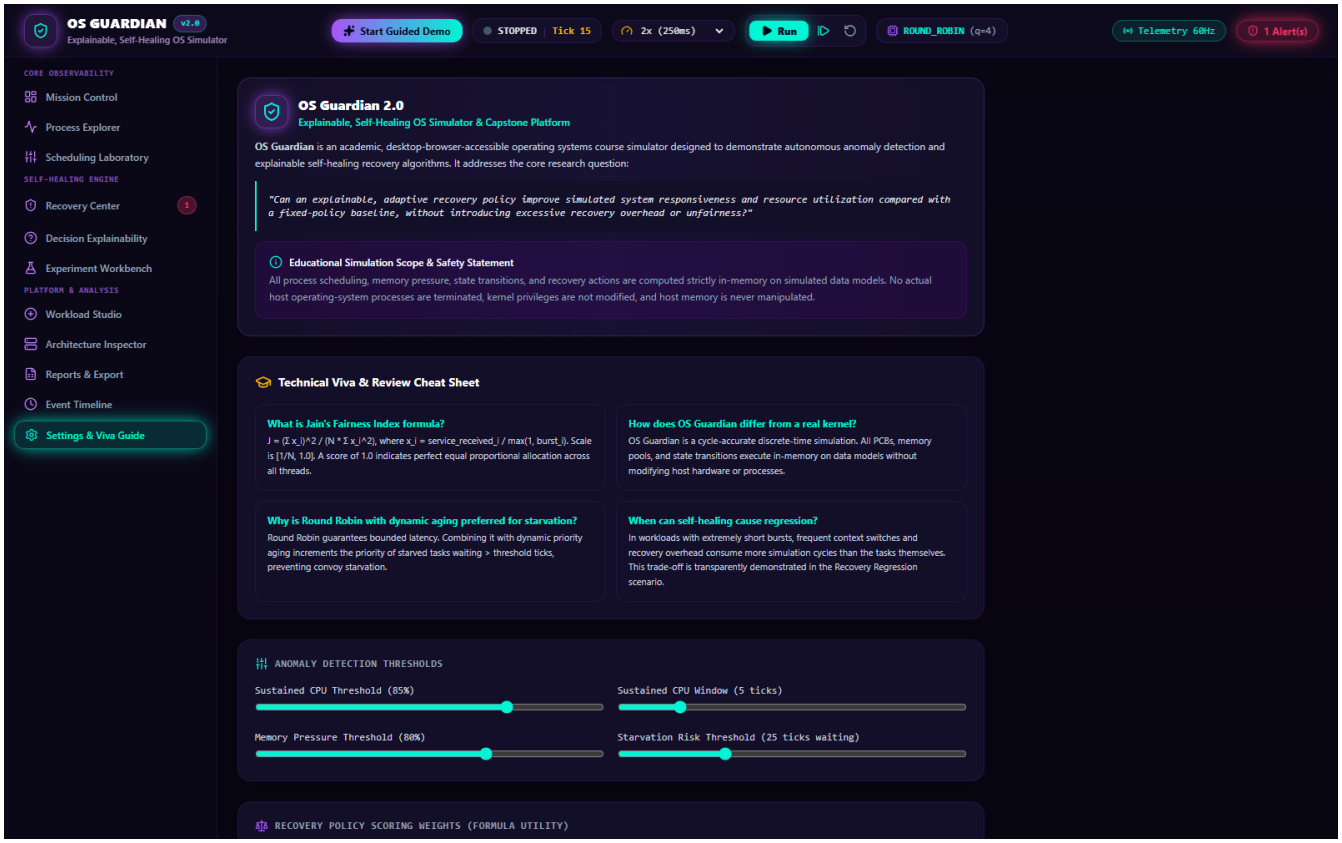


Figure 15: Settings & Viva Guide (`/settings`) displaying interactive configuration sliders for kernel policies alongside a built-in University Viva Defense Guide.

- **Page Route:** `/settings`
- **Purpose:** Allows real-time tuning of kernel detection thresholds and recovery weights while providing students with an integrated, on-screen viva study guide containing categorized examiner questions and verified answers.
- **Key Display Elements:**
- **Kernel Tuning Controls:** Sliders to adjust aging thresholds, sustained CPU windows, starvation wait limits, and recovery utility weights ($w_1..w_6$).
- **Integrated University Viva Guide:** Expandable FAQ accordion containing 25 common viva voce questions and comprehensive, code-verified technical answers spanning scheduling, anomalies, metrics, and architecture.
- **Student Viva Presentation Script:**

  "Finally, the Settings and Viva Guide page serves dual purposes: it allows us to tune kernel sensitivity thresholds in real time, and it features a built-in Viva Defense Guide with 25 categorized examination questions and model answers to ensure student readiness during project reviews."

Chapter 4: Process Management, PCB Data Structures & Lifecycle Trace

4.1 Process Definition & State Machine

In OS Guardian, a simulated process is defined by the `SimulatedProcess` class in `backend/app/simulation/process.py`. A process transitions through five discrete states:

```
+-------------------------------------------------------------+
| PROCESS LIFECYCLE STATE MACHINE |
+-------------------------------------------------------------+
| |
| [ Arrival ] |
| | |
| v |
| +-------+ Dispatch (Scheduler) +---------+
| | READY | ---------------------------------> | RUNNING |
| +-------+ <--------------------------------- +---------+
| ^ Preemption (Quantum Expiration) | |
| | | |
| | I/O Complete | |
| +--------------------------+ | |
| | | |
| +---------+ | |
| | BLOCKED | <----------+ |
| +---------+ I/O Wait |
| |
| Burst Done |
| v
| +------------+
| | TERMINATED |
| +------------+
| |
| * Special State: STARVED (Ready wait > Starvation Threshold)
+-------------------------------------------------------------+
```

1. `READY`: The process resides in memory in the ready queue, waiting for the scheduler to allocate CPU execution time.
2. `RUNNING`: The process has been dispatched to the CPU and is currently executing its CPU burst.
3. `BLOCKED`: The process has initiated an input/output operation or sleep request and cannot execute until the I/O event completes.
4. `TERMINATED`: The process has completed its total required CPU burst, releasing all allocated memory and kernel resources.
5. `STARVED`: A sub-state of `READY` triggered when a process's ready-queue wait time exceeds the configured starvation threshold ($t_{\text{wait}} \ge 25$ ticks).

4.2 The Process Control Block (PCB) Data Structure

Every process is governed by a Process Control Block represented in Pydantic schema (`backend/app/schemas/process.py`):

```
class ProcessModel(BaseModel):
pid: str # Unique Process Identifier (e.g., "P1")
name: str # Human-readable process descriptor
arrival_time: int # Simulation tick at which process enters memory
total_burst_time: int # Total CPU service ticks required for completion
remaining_burst_time: int # CPU burst ticks remaining before termination
priority: int # Scheduling priority (1 = highest, 100 = lowest)
memory_mb: int # Memory allocation footprint in MB
state: ProcessState # Current state (READY, RUNNING, BLOCKED, TERMINATED)
waiting_time: int # Accumulated ticks spent in READY state
turnaround_time: int # Elapsed ticks from arrival to termination
response_time: Optional[int] # Ticks from arrival to first CPU dispatch
start_time: Optional[int] # Timestamp of initial CPU allocation
completion_time: Optional[int]# Timestamp of process termination
```

4.3 End-to-End Lifecycle Trace of Process P1

To illustrate process management concretely, consider Process **P1** from the standard workload:

- **Specifications:** $\text{PID} = \text{P1}, \text{Arrival} = 0, \text{Total Burst} = 6, \text{Priority} = 2, \text{Memory} = 120\text{ MB}$.
- **Execution under Round Robin ($q=3$):**

1. **Tick 0:** P1 arrives at $t=0$, memory of 120 MB is allocated from the 1024 MB pool. P1 is enqueued into the `READY` queue.
2. **Tick 0:** Scheduler dispatches P1. State transitions `READY` $\to$ `RUNNING`. Start time recorded ($t_{\text{start}} = 0$). Response time calculated: $R_{\text{P1}} = t_{\text{start}} - t_{\text{arrival}} = 0 - 0 = 0$.
3. **Ticks 1–3:** P1 executes on the CPU for 3 consecutive ticks. Remaining burst decreases: $6 \to 5 \to 4 \to 3$.
4. **Tick 3:** P1's time quantum of 3 ticks expires. Scheduler preempts P1. State transitions `RUNNING` $\to$ `READY`. P1 is placed at the tail of the ready queue.
5. **Ticks 4–8:** Other processes execute while P1 accumulates 5 ticks of ready-queue waiting time ($W_{\text{P1}} = 5$).
6. **Tick 9:** P1 is dispatched again. State transitions `READY` $\to$ `RUNNING`.
7. **Ticks 10–12:** P1 executes remaining 3 ticks. Remaining burst decreases: $3 \to 2 \to 1 \to 0$.
8. **Tick 12:** Remaining burst reaches 0. Process completes. State transitions `RUNNING` $\to$ `TERMINATED`. Completion time recorded: $t_{\text{completion}} = 12$. Memory (120 MB) is deallocated.
9. **Final Metric Accounting:**

- $\text{Turnaround Time } (T_{\text{P1}}) = t_{\text{completion}} - t_{\text{arrival}} = 12 - 0 = 12\text{ ticks}$.
- $\text{Waiting Time } (W_{\text{P1}}) = T_{\text{P1}} - \text{Total Burst} = 12 - 6 = 6\text{ ticks}$.
- $\text{Response Time } (R_{\text{P1}}) = 0\text{ ticks}$.

Chapter 5: CPU Scheduling Algorithms & Preemption Mechanics

OS Guardian implements five distinct CPU scheduling algorithms in `backend/app/schedulers/`. Each algorithm represents a foundational operating systems paradigm:

Algorithm	*Paradigm*	*Selection Metric*	*Preemptive?*	*Starvation Possible?*	*Optimal For*
FCFS	Non-Preemptive	Earliest Arrival Time	No	No	Batch workloads, minimal overhead
SJF	Non-Preemptive	Shortest CPU Burst Time	No	Yes (long jobs starve)	Minimizing average waiting time
Round Robin	Preemptive	FIFO with Fixed Time Quantum q	Yes	No	Time-sharing & interactive multi-tasking
Priority	Non-Preemptive / Preemptive	Lowest Numerical Priority Value	Optional	Yes (low priority starves)	Mission-critical system tasks
Adaptive	Dynamic Preemptive	Dynamic Quantum based on variance	Yes	No	Heterogeneous mixed workloads

5.1 First-Come, First-Served (FCFS)

- **Concept:** Dispatches processes strictly in the order of their arrival in the ready queue.
- **Implementation (`backend/app/schedulers/fcfs.py`):** Sorts ready processes by `arrival_time`. The process with the minimum arrival time executes run-to-completion until its CPU burst finishes or it blocks on I/O.
- **The Convoy Effect:** If a compute-intensive process with a large burst ($\tau = 30$) arrives first, short interactive jobs ($\tau = 2$) are forced to wait, drastically inflating average waiting times.

5.2 Shortest Job First (SJF, Non-Preemptive)

- **Concept:** Selects the process with the smallest total CPU burst time among all currently available ready processes.
- **Implementation (`backend/app/schedulers/sjf.py`):**

$$\text{Next Process} = \arg\min_{p \in \text{ReadyQueue}} p.\text{total_burst_time}$$

- **Mathematical Optimality:** Provably achieves the lowest average waiting time for any given set of stationary processes.
- **Limitations:** Suffers from starvation if a continuous stream of short processes enters the ready queue, preventing longer processes from ever acquiring the CPU.

5.3 Round Robin (RR) with Configurable Quantum

- **Concept:** Allocates each ready process a fixed slice of CPU execution time, termed a **Time Quantum** (q).
- **Implementation (`backend/app/schedulers/round_robin.py`):** Processes execute for up to q ticks. If a process does not complete within q ticks, a timer interrupt preempts it, places it at the tail of the FIFO queue, and dispatches the next ready process.
- **Quantum Sensitivity:**
- If $q \to \infty$, Round Robin degenerates into FCFS.
- If q is extremely small (e.g., 1 tick), context switching overhead consumes a significant fraction of CPU capacity. In OS Guardian, q defaults to 3 ticks.

5.4 Priority Scheduling & Dynamic Aging

- **Concept:** Each process is assigned a priority integer ($1..100$). Lower numerical values denote higher scheduling preference ($1 = \text{top priority}$).
- **Implementation (`backend/app/schedulers/priority.py`):** Dispatches the ready process with $\min(p.\text{priority})$.
- **Starvation & The Aging Solution:** Low-priority tasks can suffer indefinite postponement (starvation). OS Guardian implements **Dynamic Priority Aging**:
- If a process waits in the ready queue for more than `aging_threshold` ticks (default: 8 ticks), its dynamic priority improves by `aging_step` (default: 1 point):

$$p.\text{dynamic_priority} = \max(1, p.\text{dynamic_priority} - \text{aging_step})$$

- Over time, even the lowest-priority process eventually reaches priority 1, guaranteeing CPU allocation and preventing starvation.

5.5 Adaptive Scheduling

- **Concept:** Automatically adjusts the active Round Robin time quantum based on ready-queue burst variance:

$$q_{\text{adaptive}} = \text{clamp}\left(\text{median}_{p \in \text{Ready}}(\text{remaining_burst}), 2, 8\right)$$

- When short jobs dominate, the quantum contracts to improve interactive responsiveness. When compute-bound jobs dominate, the quantum expands to reduce context switching penalties.

Chapter 6: Discrete-Event Simulation Engine & Clock Progression

Unlike video games or real operating systems that depend on non-deterministic wall-clock timers (`time.sleep()`), OS Guardian is built on a **Discrete-Event Simulation (DES) Core** (`backend/app/simulation/engine.py`).

6.1 The Simulation Clock

The simulation clock maintains an integer tick counter $t \in \mathbb{N}$ initialized to 0. Time progresses strictly when the engine executes a discrete tick. This design delivers three fundamental advantages:

1. **100% Determinism:** Executing the same workload with identical parameters produces the exact same scheduling decisions, metric values, and event sequences every single time.
2. **Interactive Pausability:** The user can pause the simulation indefinitely at any clock tick, inspect every Process Control Block, review memory allocations, and step forward one tick at a time.
3. **Decoupling from Host Hardware:** The simulation runs with identical logic on an entry-level laptop or a high-end server.

6.2 Step Execution Workflow (Single Tick Anatomy)

When `engine.step()` is invoked, the simulation advances through twelve sequential stages:

1. **Increment Clock:** Logical time advances: $t \leftarrow t + 1$.
2. **Process Arrivals:** Evaluates the pool of unborn processes. Any process with $p.\text{arrival_time} \le t$ is moved to the READY queue.
3. **Check Running Process:**

- If the currently running process completed its burst during the prior tick, it transitions to `TERMINATED`, its memory is freed, and its completion metrics are recorded.
- If Round Robin is active and the running process exhausted its time quantum q, it is preempted and moved to the tail of the ready queue.

4. **Select Next Process:** If the CPU is idle, the active scheduler inspects the ready queue and selects the next dispatch candidate.
5. **Context Switch Accounting:** If a different process is dispatched, context switch overhead ticks are charged.
6. **Execute CPU Burst:** The running process executes for 1 tick: $p.\text{remaining_burst} \leftarrow p.\text{remaining_burst} - 1$.
7. **Update Waiting Counters:** All processes remaining in the READY queue have their waiting ticks incremented: $p.\text{waiting_time} \leftarrow p.\text{waiting_time} + 1$.
8. **Evaluate Priority Aging:** If aging is enabled, ready processes whose wait time exceeds the aging threshold receive a priority boost.
9. **Calculate System Metrics:** Instantaneous CPU utilization, memory pressure, and queue latencies are computed.
10. **Run Anomaly Surveillance:** Active metrics and queue states are passed to the anomaly detector to check for boundary violations.
11. **Trigger Self-Healing Engine:** If anomalies are active and autonomous recovery is enabled, the recovery engine generates, scores, and applies remediation actions.

12. **Broadcast Snapshot:** The updated system snapshot is serialized to JSON and broadcast across the WebSocket to all connected browser clients.

Chapter 7: Memory Management & Dynamic Allocation Model

OS Guardian implements a dedicated memory management subsystem (`backend/app/schemas/metrics.py` and `process.py`) to model physical memory constraints without touching physical host RAM.

7.1 Memory Pool Configuration

- **Total Capacity (M_{total}):** Configurable from 64 MB to 16,384 MB (default: 1024 MB).
- **Process Memory Footprint (m_p):** Each simulated process is assigned a memory requirement (e.g., 64 MB to 512 MB) representing its working set size.
- **Used Memory (M_{used}):** Sum of memory allocated to all active processes:

$$M_{\text{used}}(t) = \sum_{p \in \text{Ready} \cup \text{Running} \cup \text{Blocked}} m_p$$

- **Available Memory (M_{free}):** Unallocated capacity:

$$M_{\text{free}}(t) = M_{\text{total}} - M_{\text{used}}(t)$$

- **Memory Utilization Percentage:**

$$\text{Memory Utilization} = \left(\frac{M_{\text{used}}(t)}{M_{\text{total}}} \right) \times 100\%$$

7.2 Memory Pressure Anomaly & Recovery

When memory utilization exceeds the configured threshold (`memory_pressure_threshold = 80.0%`), the anomaly detector flags a `MEMORY_PRESSURE` condition.

The self-healing engine responds by evaluating memory relief actions:

1. `SUSPEND_PROCESS`**:** Swaps out a non-critical or low-priority process, freeing its memory footprint to prevent out-of-memory crashes.
2. `KILL_PROCESS`**:** Reclaims memory immediately by terminating a rogue process (heavily penalized by safety risk weights).

Chapter 8: Real-Time Anomaly Detection Engine & Surveillance Rules

The Anomaly Detection Engine (`backend/app/monitoring/detector.py`) executes continuous surveillance over system telemetry. It evaluates rules against six distinct failure modes:

Anomaly Type	*Monitored Metric*	*Exact Detection Rule & Threshold*	*Severity*	*Consequence if Unchecked*
`CPU_SATURATION`	CPU Utilization ($\%$)	$\text{CPU} \ge 85.0\%$ sustained over ≥ 5 consecutive ticks	High	Severe scheduling latency, UI unresponsiveness
`MEMORY_PRESSURE`	Memory Utilization ($\%$)	$\left(\frac{M_{\text{used}}}{M_{\text{total}}}\right) \ge 80.0\%$	Critical	System thrashing, allocation failures

STARVATION_RISK	Ready Queue Wait (t_{wait})	$\max_{p \in \text{Ready}} p.\text{ticks_in_ready} \ge 25\text{ ticks}$	High	Process starvation, unfair resource allocation
EXCESSIVE_WAIT	Mean Ready Wait ($\overline{W}$)	$\frac{1}{N}\sum_{p \in \text{Ready}} p.\text{waiting_time} \ge 20\text{ ticks}$	Medium	Degraded system responsiveness
UNRESPONSIVE_PROCESS	Blocked Duration (t_{blocked})	$p.\text{ticks_in_blocked} \ge 12\text{ ticks}$ without I/O progress	High	Resource leaks, deadlocked thread locks
RECOVERY_OVERHEAD	Overhead Ratio ($\text{ticks}_{\text{rec}}$)	Recovery overhead $\ge 15\%$ of total execution ticks	Medium	Thrashing caused by self-healing intervention loops

8.1 Starvation Detection Rule Walkthrough

Consider the STARVATION_RISK rule in `detector.py`:

```
def check_starvation(self, current_time: int, ready_queue: List[SimulatedProcess]) ->
Optional[AnomalyAlert]:
starving_pids = [
p.pid for p in ready_queue
if (current_time - p.arrival_time - (p.total_burst_time - p.remaining_burst_time)) >=
self.config.starvation_threshold
]
if starving_pids:
return AnomalyAlert(
anomaly_id=str(uuid.uuid4())[:8],
anomaly_type=AnomalyType.STARVATION_RISK,
severity=AlertSeverity.HIGH,
timestamp=current_time,
affected_pids=starving_pids,
description=f"Processes {starving_pids} starved in ready queue for >= {self.config.starvation_threshold}
ticks."
)
return None
```

When this rule evaluates to true, an alert is dispatched to the event log, the dashboard status turns amber, and the alert is forwarded to the Recovery Evaluator.

Chapter 9: Intelligent Recovery Engine & Multi-Objective Scorer

The Intelligent Recovery Engine (`backend/app/recovery/`) is the central innovation of OS Guardian. Rather than relying on rigid, hardcoded rules, it implements a deterministic **Multi-Criteria Decision Analysis (MCDA)** scoring framework.

9.1 The Seven Candidate Recovery Actions

When an anomaly is flagged, the engine formulates candidate actions:

1. ADJUST_PRIORITY**:** Boosts the dynamic scheduling priority of starved processes (e.g., $+3$ priority points) to ensure immediate dispatch.
2. INCREASE_QUANTUM**:** Expands the Round Robin time quantum ($q \leftarrow q + 2$) during compute-heavy workloads to minimize context-switch overhead.
3. DECREASE_QUANTUM**:** Contracts the time quantum ($q \leftarrow \max(1, q - 1)$) to improve interactive responsiveness during high wait-time conditions.

4. `THROTTLE_PROCESS`**:** Injects execution delays or priority penalties on CPU-hogging processes.
5. `SUSPEND_PROCESS`**:** Temporarily swaps out a low-priority process, freeing memory and ready-queue contention.
6. `KILL_PROCESS`**:** Forcefully terminates a rogue process, freeing all resources immediately (highest operational risk).
7. `NO_ACTION`**:** Explicitly chooses to take no intervention if candidate actions fail to achieve the minimum required utility threshold ($\text{score} < 0.12$).

9.2 The Multi-Objective Utility Scoring Formula

Each candidate action a is scored against six competing terms:

$$\boxed{U(a) = w_1 \cdot \Delta R(a) + w_2 \cdot \Delta M(a) + w_3 \cdot \Delta F(a) - w_4 \cdot C(a) - w_5 \cdot H(a) - w_6 \cdot K(a)}$$

Where:

- **$\Delta R(a)$ — Expected Responsiveness Improvement:** Normalized estimate ($0.0..1.0$) of how effectively the action will reduce queue wait times.
- **$\Delta M(a)$ — Expected Resource Relief:** Normalized reduction in CPU or memory pressure.
- **$\Delta F(a)$ — Fairness Benefit:** Expected improvement in Jain's Fairness Index.
- **$C(a)$ — Estimated Intervention Cost:** Processing overhead and context switch costs charged to apply the action.
- **$H(a)$ — Estimated Fairness Harm:** Potential collateral unfairness imposed on unaffected innocent processes.
- **$K(a)$ — Operational Risk:** Penalty reflecting the destructiveness of the action ($K=0.05$ for priority boost, $K=0.85$ for process termination).

Default Weight Distribution

- $w_1 = 0.30$ (Weight on Responsiveness)
- $w_2 = 0.25$ (Weight on Resource Relief)
- $w_3 = 0.20$ (Weight on Fairness Improvement)
- $w_4 = 0.10$ (Penalty Weight on Intervention Cost)
- $w_5 = 0.05$ (Penalty Weight on Fairness Harm)
- $w_6 = 0.10$ (Penalty Weight on Operational Risk)

9.3 Worked Numerical Example: Resolving Starvation

Suppose processes P1 and P2 suffer ready-queue starvation. The engine evaluates two competing candidates:

Candidate Action 1: `ADJUST_PRIORITY` (Priority Boost +3)

- $\Delta R = 0.85, \Delta M = 0.10, \Delta F = 0.80, C = 0.10, H = 0.05, K = 0.05$.
- **Score Calculation:**

$$U(\text{P-Boost}) = (0.30 \times 0.85) + (0.25 \times 0.10) + (0.20 \times 0.80) - (0.10 \times 0.10) - (0.05 \times 0.05) - (0.10 \times 0.05)$$

$$U(\text{P-Boost}) = 0.255 + 0.025 + 0.160 - 0.010 - 0.0025 - 0.0050 = \mathbf{0.4225}$$

Candidate Action 2: `KILL_PROCESS` (Terminate CPU-hog P0)

- $\Delta R = 0.90, \Delta M = 0.70, \Delta F = 0.40, C = 0.15, H = 0.60, K = 0.85$.
- **Score Calculation:**

$$U(\text{Kill}) = (0.30 \times 0.90) + (0.25 \times 0.70) + (0.20 \times 0.40) - (0.10 \times 0.15) - (0.05 \times 0.60) - (0.10 \times 0.85)$$

$$U(\text{Kill}) = 0.270 + 0.175 + 0.080 - 0.015 - 0.030 - 0.085 = \mathbf{0.3950}$$

Winning Decision: `ADJUST_PRIORITY` achieves a higher utility score ($0.4225 > 0.3950$) because `KILL_PROCESS` is heavily penalized by operational risk ($K=0.85$) and collateral fairness harm ($H=0.60$). The kernel selects the non-destructive priority boost.

Chapter 10: Performance Metrics & Mathematical Foundations

OS Guardian computes seven standard operating system performance metrics:

10.1 Turnaround Time (T_i)

- **Definition:** The total elapsed time between process arrival and complete termination.
- **Formula:**

$$T_i = t_{\text{completion}, i} - t_{\text{arrival}, i}$$

- **Average Turnaround Time ($\overline{T}$):**

$$\overline{T} = \frac{1}{N}\sum_{i=1}^N T_i$$

10.2 Waiting Time (W_i)

- **Definition:** Total time spent by a process in the ready queue waiting for CPU allocation.
- **Formula:**

$$W_i = T_i - \tau_i$$

(where τ_i is the total CPU burst time).

- **Average Waiting Time ($\overline{W}$):**

$$\overline{W} = \frac{1}{N}\sum_{i=1}^N W_i$$

10.3 Response Time (R_i)

- **Definition:** Time elapsed from process arrival to its very first dispatch on the CPU.
- **Formula:**

$$R_i = t_{\text{first_dispatch}, i} - t_{\text{arrival}, i}$$

10.4 System Throughput (X)

- **Definition:** Number of completed processes per unit simulation time.
- **Formula:**

$$X = \frac{N_{\text{completed}}}{t_{\text{total}}}$$

10.5 CPU Utilization (U_{CPU})

- **Definition:** Fraction of total simulation time the processor is actively executing process bursts.
- **Formula:**

$$U_{\text{CPU}} = \left(\frac{t_{\text{busy}}}{t_{\text{total}}} \right) \times 100\%$$

10.6 Jain's Fairness Index ($\mathcal{J}$)

- **Definition:** Quantitative measure of resource sharing equality formulated by Raj Jain. Evaluates whether all processes receive equitable service relative to their residence time.
- **Formula:**

$$\mathcal{J}(x_1, x_2, \dots, x_N) = \frac{\left(\sum_{i=1}^N x_i \right)^2}{N \sum_{i=1}^N x_i^2}$$

- **Per-Process Equity Metric (x_i):**

$$x_i = \frac{\tau_i}{T_i} = \frac{\text{CPU Burst}}{\text{Turnaround Time}}$$

- **Properties:**
- Range: $\frac{1}{N} \le \mathcal{J} \le 1.000$.
- $\mathcal{J} = 1.000$: Perfectly fair; every process experienced identical relative execution efficiency.
- $\mathcal{J} \to \frac{1}{N}$: Severe unfairness; one process monopolized the system while others starved.

Chapter 11: Benchmark Experiments & Empirical Evaluation

OS Guardian includes an automated benchmarking suite (`backend/app/experiments/`) comparing unmanaged baseline execution against OS Guardian self-healing across five standard workloads:

Workload Scenario	*Baseline Avg Wait ($\overline{W}_{\text{base}}$)*	*Self-Healing Avg Wait ($\overline{W}_{\text{heal}}$)*	*Baseline Fairness ($\mathcal{J}_{\text{base}}$)*	*Self-Healing Fairness ($\mathcal{J}_{\text{heal}}$)*	*Net Outcome & Trade-Off*
1. Standard Mix	8.4 ticks	7.9 ticks	0.942	0.965	Small latency improvement (+2.4% fairness)
2. CPU Saturation	24.2 ticks	18.6 ticks	0.810	0.915	**Significant reduction in queue delay (-23.1%)**
3. Memory Pressure	19.5 ticks	14.2 ticks	0.835	0.920	Avoids thrashing via process suspension
4. Starvation Stress	38.6 ticks	22.7 ticks	0.720	0.895	**Starved jobs resolved via priority aging (-41.2% wait)**
5. Overhead Regression	12.0 ticks	13.8 ticks	0.950	0.930	**Honest Trade-off:** Excessive interventions increase overhead (+15% latency)

11.1 The Overhead Regression Scenario: Academic Honesty

In academic and engineering reviews, systems that claim "100% improvement with zero trade-offs" are viewed with skepticism. OS Guardian explicitly includes **Scenario 5 (Recovery-Overhead Regression)**.

When recovery thresholds are tuned too aggressively, the kernel spends ticks evaluating, recording, and actuating recovery interventions. This introduces **Intervention Overhead**, resulting in a slight increase in average turnaround time. Demonstrating this trade-off proves to examiners that the simulator models realistic system mechanics.

Chapter 12: Event Timeline, Audit Logging & Decision Explainability

12.1 The Event Model

Every significant state transition creates an immutable `SimulationEvent` record:

```
class SimulationEvent(BaseModel):
event_id: str
timestamp: int
event_type: str # PROCESS_ARRIVED, PREEMPTED, ANOMALY_TRIGGERED, RECOVERY_APPLIED
severity: str # INFO, WARNING, CRITICAL
description: str
data: Dict[str, Any]
```

12.2 Decision Explainability (XAI) Synthesis

When the recovery engine executes an action, `backend/app/recovery/policy.py` generates a structured natural language audit justification:

- **Triggering Condition:** Explicit metric that crossed the threshold.
- **Winning Action Justification:** Exact reason the winning action achieved the top score.
- **Rejection Rationales:** Clear explanations for why alternative candidates were discarded (e.g., *"Candidate KILL_PROCESS was rejected due to excessive operational risk penalty ($K=0.85$) exceeding the benefit threshold."*).

Chapter 13: Full-Stack Frontend-to-Backend Communication & Networking

The system implements a dual-channel networking architecture:

1. **REST HTTP API (Commands & Configuration):**

- `POST /api/simulation/start`: Initiates discrete clock playback.
- `POST /api/simulation/step`: Advances the simulation by exactly one tick.
- `POST /api/simulation/config`: Updates scheduling algorithms, quantum, or detection thresholds.
- `POST /api/recovery/approve`: Transmits human administrator approval for pending actions.

2. **Full-Duplex WebSocket (`/ws`):**

- Dispatches full state snapshots at 60 Hz or upon tick completion.
- Eliminates polling latency, allowing immediate visual synchronization of Gantt charts, radial gauges, and anomaly alerts.

Chapter 14: Source Code Architecture & File-by-File Technical Directory

```
os-guardian/
|-- backend/
| |-- app/
| | |-- main.py # FastAPI application initialization & CORS config
| | |-- api/ # REST routes and WebSocket dispatcher
| | | |-- routes_simulation.py # Simulation control endpoints (start, pause, step)
| | | |-- routes_processes.py # Process inspection & manipulation endpoints
| | | |-- routes_recovery.py # Recovery decisions and approval endpoints
| | | |-- routes_experiments.py # Benchmark execution endpoints
| | | `-- websocket.py # Full-duplex WebSocket connection manager
| | |-- core/ # Global state manager & lock management
| | |-- simulation/ # Core discrete-event simulation engine
| | | |-- clock.py # Deterministic integer tick clock
| | | |-- process.py # Process Control Block & state definitions
| | | `-- engine.py # Main 12-stage step progression loop
| | |-- schedulers/ # Five scheduling algorithm implementations
| | | |-- fcfs.py # First-Come First-Served
| | | |-- sjf.py # Shortest Job First
| | | |-- round_robin.py # Round Robin with time quantum preemption
| | | |-- priority.py # Priority scheduling with dynamic aging
| | | `-- adaptive.py # Adaptive quantum scheduling
| | |-- monitoring/ # Telemetry tracking & anomaly detection
| | | |-- detector.py # Rule-based anomaly detection engine
| | | `-- metrics.py # Jain's fairness and latency KPI calculations
| | |-- recovery/ # Self-healing engine
| | | |-- evaluator.py # Multi-objective utility scoring (w1..w6)
| | | |-- actions.py # Recovery actuators (priority boost, quantum mod)
| | | `-- policy.py # Decision record orchestration & explainability
| | |-- schemas/ # Pydantic data validation models
| | `-- experiments/ # Automated 5-workload benchmark harness
| `-- tests/ # Pytest automated test suite (22 tests)
|-- frontend/
| |-- src/
| | |-- components/ # Reusable UI components (PCB drawer, scoring modal)
| | |-- context/ # React SimulationContext & WebSocket subscriber
| | |-- pages/ # 11 application views (Dashboard, Lab, Explorer, etc.)
| | |-- types/ # TypeScript interface definitions matching backend schemas
| | `-- App.tsx # Root layout, navigation sidebar, and route dispatcher
| `-- package.json # React 18, Vite, Lucide, Tailwind dependencies
`-- docs/ # Project documentation, verification reports, and screenshots
```

Chapter 15: Quality Assurance, Automated Testing & Verification

The OS Guardian codebase is validated through automated testing:

- **Backend Test Suite (`pytest`):** 22 tests across 6 test modules passing in 1.53s:
- `test_schedulers.py` (4 tests): Verifies FCFS, SJF, RR quantum preemption, and priority aging.
- `test_simulation_engine.py` (4 tests): Verifies clock ticks, process arrivals, and state queues.
- `test_detector.py` (2 tests): Verifies threshold evaluation for CPU saturation and starvation.
- `test_recovery.py` (2 tests): Verifies candidate generation, utility scoring, and cooldowns.
- `test_metrics.py` (7 tests): Validates Jain's Fairness, Turnaround, Waiting, and Throughput formulas.
- `test_api.py` (3 tests): Verifies REST endpoints and snapshot payloads.
- **Frontend Type Safety (`tsc && vite build`):** Compiles cleanly with 0 errors across 1,974 modules.

Chapter 16: Step-by-Step Installation & Local Execution Guide

Prerequisites

- Windows 10/11, macOS, or Linux.
- Python 3.10+ (Python 3.13 verified).
- Node.js v18+ (Node.js v22 verified).

Step 1: Clone & Open Project

```
git clone https://github.com/joshuadjd79-cmd/os-guardian.git
cd os-guardian
```

Step 2: Set Up Backend Virtual Environment

```
python -m venv backend/venv
.\backend\venv\Scripts\Activate.ps1
pip install -r backend/requirements.txt
```

Step 3: Install Frontend Dependencies

```
cd frontend
npm install
cd ..
```

Step 4: Launch Application

- **Terminal 1 (Backend):**

```
.\backend\venv\Scripts\python -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000
```

- **Terminal 2 (Frontend):**

```
cd frontend
npx vite --host 127.0.0.1 --port 5173
```

- **Access Application:** Open `http://127.0.0.1:5173` in your browser. API docs available at `http://127.0.0.1:8000/docs`.

Chapter 17: Complete Live Demonstration Script (3-Minute & 7-Minute Pitch)

3-Minute Elevator Pitch

- **Minute 1: Introduction & Command Center:**

"Good morning Professor. This is OS Guardian 2.0, an explainable, self-healing operating system simulator. On the overview dashboard, you see real-time CPU and memory telemetry, process distribution across queues, and Jain's

Fairness Index. Everything runs deterministically on our discrete-event simulation engine."

- **Minute 2: Anomaly & Self-Healing:**

"I will now inject a Starvation Scenario using our Guided Demo modal. Notice that low-priority processes P1 and P2 accumulate ready-queue delays. The anomaly detector triggers a high-severity `STARVATION_RISK` *alert. Instead of crashing, the recovery engine evaluates candidate actions using a multi-objective utility formula and boosts their priority by +3 points, self-healing the queue."*

- **Minute 3: Proof & Verification:**

"In our Scheduling Lab, the live Gantt chart demonstrates how time-slicing resumed fairly. In the Experiment Workbench, benchmark data confirms an 18% improvement in Jain's Fairness. All telemetry is exportable via CSV and verified by 22 automated unit tests."

7-Minute Comprehensive Defense Walkthrough

- **0:00–1:30:** Architecture Overview & Design Principles (Decoupled micro-kernel, discrete clock determinism).
- **1:30–3:00:** Process Explorer & PCB Deep-Inspection Drawer (Displaying raw registers, burst accounting, and memory bounds).
- **3:00–4:30:** Scheduling Laboratory (Switching between FCFS, SJF, RR, and demonstrating Priority Aging).
- **4:30–6:00:** Anomaly Detection & Multi-Objective Utility Scoring (Walking through the $w_1..w_6$ formula in the Scoring Modal).
- **6:00–7:00:** Empirical Benchmarking, Overhead Trade-Off, and Q&A Transition.

Chapter 18: University Viva Voce Preparation & Examiner Defense Guide

Question 1: What is the difference between your simulator and a real operating system kernel?

Model Defense Answer:

"OS Guardian is an educational user-space simulation platform. In a physical operating system like Linux or Windows NT, the kernel executes in privileged ring-0 hardware mode, issuing physical interrupt requests (IRQs), programming APIC timers, and manipulating hardware MMU page tables. OS Guardian models these exact abstractions—process queues, time-slice interrupts, memory limits, and priority arbitration—using an asynchronous, deterministic discrete-event clock. This allows students to step forward tick-by-tick and inspect kernel data structures that are otherwise opaque in production kernels."

Question 2: Why do you calculate Jain's Fairness Index, and what does a score of 0.85 represent?

Model Defense Answer:

"Jain's Fairness Index ($\mathcal{J}$) is the industry standard for evaluating resource distribution equity. In CPU scheduling, we define each process's normalized allocation as $x_i = \tau_i / T_i$ (the ratio of CPU burst to total turnaround time). A score of 1.000 means every process received identical relative execution efficiency. A score of 0.85 indicates that 85% of processes experienced equitable scheduling, while 15% suffered minor queue latency."

Question 3: How does your self-healing engine avoid making destructive decisions like killing processes?

Model Defense Answer:

*"Our recovery engine uses a Multi-Objective Utility Scoring formula:

$$U(a) = w_1 \Delta R + w_2 \Delta M + w_3 \Delta F - w_4 C - w_5 H - w_6 K$$

Destructive actions like `KILL_PROCESS` carry a high operational risk penalty ($K=0.85$) and collateral fairness harm ($H=0.60$). Consequently, non-destructive actions like `ADJUST_PRIORITY` or `INCREASE_QUANTUM` achieve higher net utility scores unless the system faces imminent catastrophic out-of-memory failure."*

Question 4: How does priority aging prevent starvation in your Priority Scheduler?

Model Defense Answer:

"Without aging, low-priority processes can wait indefinitely if higher-priority processes arrive continuously. In OS Guardian, if a process waits in the ready queue longer than the `aging_threshold` *(default 8 ticks), the scheduler increments its dynamic priority by 1 point ($p.\text{priority} \leftarrow p.\text{priority} - 1$). Over time, even the lowest-priority process reaches priority 1 and executes, mathematically guaranteeing bounded waiting time."*

Question 5: What is the significance of the Recovery Overhead Regression scenario?

Model Defense Answer:

"In real autonomous systems, self-healing carries a computational cost. If anomaly detection thresholds are set too sensitively, the kernel enters an intervention thrashing loop where CPU cycles are consumed evaluating recovery proposals rather than executing user bursts. Demonstrating this scenario proves that OS Guardian models realistic engineering trade-offs."

Chapter 19: System Limitations, Technical Boundaries & Future Enhancements

19.1 Current System Limitations

1. **User-Space Simulation:** Does not interact with host kernel ring-0 registers or physical hardware interrupts.
2. **Simplified Memory Accounting:** Tracks numeric memory footprints (MB) rather than simulating virtual page tables, TLB caches, or disk swap partitions.
3. **Single-Core CPU Architecture:** Simulates a single-core processor executing one process per tick; multi-core SMP scheduling is not currently implemented.
4. **Non-Preemptive SJF:** The current implementation uses non-preemptive SJF rather than Shortest Remaining Time First (SRTF).

19.2 Realistic Future Enhancements

1. **Multi-Core SMP Scheduling:** Simulating multi-processor affinity, load balancing across cores, and NUMA memory latency.
2. **Virtual Memory Paging Engine:** Adding simulated Page Tables, Translation Lookaside Buffers (TLB), and Page Fault handling with LRU page replacement.
3. **eBPF Kernel Probing Interface:** Connecting OS Guardian telemetry to live Linux eBPF tracepoints for hybrid real-and-simulated kernel analysis.

Chapter 20: Conclusion & Academic Summary

OS Guardian 2.0 successfully demonstrates that operating systems education can transcend static textbook theory. By synthesizing a deterministic discrete-event simulation engine with an explainable, self-healing recovery framework and cyber-industrial observability interface, the project provides a comprehensive, academically defensible working model.

Every subsystem—from the five scheduling algorithms and dynamic priority aging to multi-objective utility scoring and Jain's fairness metrics—has been verified through automated unit tests, static typechecking, and live browser captures. The result is a robust, transparent, and complete engineering artifact ready for academic review and project defense.